

SAPIENZA
UNIVERSITÀ DI ROMA

"Sapienza" University of Rome
Faculty of Information Engineering,
Informatics and Statistics
Department of Computer Science

ADM

Algorithmic Methods of Data Mining

Author
Matteo

October 26, 2026

Acknowledgements

Thanks to Flavia for sharing her handwritten notes for Algorithmic Methods of Data Mining, which made this document possible.

Contents

1 Introduction to data mining **1**

1.1 Course organization 1

1.2 What is data mining? 1

1.3 Structured, unstructured and semi-structured data 2

1.4 Types of data 2

1.5 Data as mathematical objects 3

2 Analysis of algorithms **5**

2.1 Problems and algorithms 5

2.2 Correctness 6

2.3 Efficiency: counting steps 6

2.4 Asymptotic notation 7

2.5 Lower bounds and optimal algorithms 10

2.6 Prefix sums 11

2.7 An open gap: matrix multiplication 12

3 Recursion **14**

3.1 Fibonacci numbers 14

3.2 Binary search 17

4 Complexity classes **20**

4.1 The traveling salesman problem 20

4.2 Decision problems, P and NP 21

4.3 Reductions and NP-completeness 22

4.4 Some NP-complete problems 24

4.5 NP-hard problems 26

5 Approximation and heuristics **27**

5.1 Approximation algorithms 27

5.2 Heuristics 28

6 Distance measures **31**

6.1 Why distances? 31

6.2 Distance functions 31

6.3 L_p distances 32

6.4 Distances between sets 33

7 Distances between strings **36**

7.1 Why compare strings? 36

7.2 First attempts 36

7.3 Subsequences 37

7.4 Edit distance 38

7.5 A recursive formula for the edit distance 39

8 Memory and data representation **42**

8.1 Bytes, addresses and consecutive memory locations 42

8.2 Python names refer to objects 43

9 Text mining **45**

9.1 From raw text to terms 45

9.2 From documents to vectors 46

9.3 Why document length can distort a comparison 47

9.4 Removing the effect of length: normalization 48

9.5 Comparing directions: cosine similarity 49

9.6 Why some words should count more than others 51

9.7 Weighting terms by their rarity: IDF 52

9.8 Combining term frequency and rarity: TF-IDF 53

9.9 Using document vectors for search 55

9.10 Boolean AND search 57

9.11 The inverted index 58

9.12 Intersecting sorted lists with pointers 59

10 Sorting **64**

10.1 The sorting problem 64

10.2 MergeSort 65

10.3 MergeSort: split, sort the halves, and merge 68

11 Randomized Algorithms **72**

11.1 Making random choices inside an algorithm 72

11.2 Expected value: averaging over random outcomes 72

11.3 QuickSort: choose a pivot and partition 73

11.4 The cost of QuickSort 76

1 Introduction to data mining

Start of lecture 24 September

1.1 Course organization

The course has two parts. The first part brings everyone to the same level, especially those who have not studied computer science before: basic computer architecture, data structures (different problems call for different ways of storing data), algorithms (how to solve a problem quickly, or even optimally, and how to measure its complexity), and programming, practised in the lab, in the homeworks and in the hackathon. The second part covers data mining topics: text mining, clustering and graphs.

1.2 What is data mining?

Huge amounts of data are produced in every field: social networks, for instance, both contain and generate an enormous quantity of data. These data hide patterns that we would like to find, but the amount is far too large to inspect by hand. Data mining tools let us explore data that we could not otherwise process.

Definition 1.1: Data mining

Data mining is the use of efficient techniques for the analysis of very large collections of data and the extraction of useful and possibly unexpected patterns in data.

We will meet two kinds of tasks:

- **Descriptive tasks** summarize the data and reveal their structure, for example groups of similar objects.

- **Predictive tasks** use the data to predict something unknown, for example whether a person will click on an advertisement.

1.3 Structured, unstructured and semi-structured data

Data come with different amounts of organization.

Definition 1.2: Structured, unstructured and semi-structured data

- *Structured data* have a clean, fixed form. For example, a university's student records store a name, a surname and a student ID for every student. These are the best data we can hope for.
- *Unstructured data* have no predefined form, for example text and pages found on the internet. Most of the data we meet are of this kind.
- *Semi-structured data* lie in between: they have some structure, but it can be modified freely. Spreadsheets (Excel) and XML files are typical examples.

Note: Data cleaning

Data must be cleaned before they are analysed: if the data are dirty, the algorithm will not work. Cleaning used to be very hard to do by hand; today software tools help, but we should still check that the result is correct. Semi-structured data are especially prone to errors, because their structure is easy to break.

Start of lecture 28 September

1.4 Types of data

Data are very heterogeneous. Common types include:

- **Table (record) data**, such as spreadsheets; also images and time series.
- **Transaction data**: each record is a transaction between two parties, such as a credit-card payment or a shopping basket.
- **Document data**: Wikipedia pages, tweets, ...
- **Network data**: the web, where pages are linked to one another. Google works with these graphs, and Wikipedia's internal links form one as well.
- **Genomic sequences**: databases of DNA sequences, often used to find correlations with diseases.
- **Climate data**.
- **Behavioural data**: everything recorded about us, for example by smartphones.

1.5 Data as mathematical objects

1.5.1 Records, attributes and sparsity

In table data, each *row* describes an object (for example a person) and each *column* describes an *attribute* of the object.

Definition 1.3: Sparsity

A data table is *sparse* when most of its cells are empty (or zero). The *sparsity* of a table describes how many of its cells are filled, that is, whether many or few values are missing.

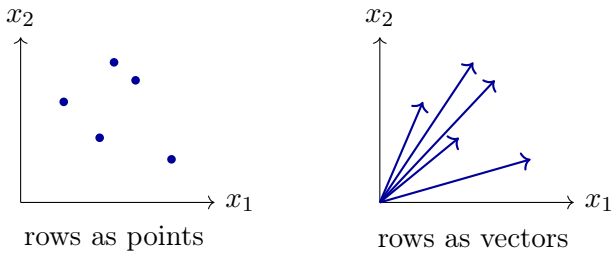
Attributes can be of different types:

- **Binary**: two possible values, such as yes/no.
- **Categorical**: values in a finite set of classes, such as a colour or a school grade like "good" or "bad".
- **Numeric**: numbers, either *discrete* (counts) or *continuous* (time, length, temperature).

We can also move between types: a numeric attribute can be turned into a categorical one by grouping values into classes, and categories can be encoded as numbers.

1.5.2 Numeric records as points and vectors

To analyse data with algorithms, we represent them as mathematical objects. Suppose a table has 2000 rows (people) and 7 numeric attributes. Each row is then an element of $\mathbb{R}^7$, which we can view either as a *point* in a 7-dimensional space or as a *vector* starting from the origin.



1.5.3 Document data

A text contains many words. We represent a document as a vector with one dimension per word of the vocabulary; each entry counts how many times that word appears in the document.

Example 1: Documents as word-count vectors

	word 1	word 2	word 3	⋯
Document 1	10	7	1	⋯
Document 2	15	5	0	⋯

The vocabulary is large, and each document uses only a small part of it. Most entries are therefore 0, so document data have a very high sparsity.

1.5.4 Transaction data

Each transaction has an identifier (TID) and the set of items it involves.

Example 2: A transaction table

TID	Items
1	bread, coke, milk
2	milk, apples
3	carrots
4	chocolate, honey

1.5.5 Ordered data and graph data

In *ordered data* the order of the entries is essential. A genomic sequence such as `GGTTCCGCTTCA`... is an ordered string of characters; time series are another example. In *graph data*, objects are nodes and relations are edges. The classic example is the web graph, whose nodes are pages and whose edges are links.

2

Analysis of algorithms

2.1 Problems and algorithms

A computational *problem* specifies which inputs are allowed and which output each input should produce. An *algorithm* is a finite sequence of steps that produces the required output for every input. Our first problem is finding the maximum of an array.

Problem: Max-Find

Input: an array $A = (A[1], A[2], \ldots, A[n])$ of n numbers.
Output: the maximum value in A.

Note: Indexing

In pseudocode we index arrays from 1, so $A[i]$ is the i-th element. In Python, indexing starts from 0: for $\vec{x} = (x_1, x_2, x_3)$ we have `x[0]` $= x_1$. The length of the array is `n = len(A)`.

Algorithm 2.1: Max-Find

```
1: function MaxFind(A)
2:     max ← A[1]
3:     for i = 1 to n do
4:         if max < A[i] then
5:             max ← A[i]
6:         end if
7:     end for
8:     return max
9: end function
```

The loop may also start from $i = 2$, since comparing $A[1]$ with itself is useless. For every algorithm we always ask two questions:

1. **Correctness:** does it solve my problem?
2. **Efficiency:** how fast is it?

2.2 Correctness

Testing an algorithm on some inputs is not enough: it may work only on the inputs we tried. To show that an algorithm is *wrong*, a single counterexample suffices. To show that it is *correct*, we need a proof, typically by induction on the number of steps performed.

Proposition 2.1: Correctness of Max-Find

After the iteration with index i, the variable max in Algorithm 2.1 equals

$$\max\{A[1], \ldots, A[i]\}.$$

In particular, when the loop ends ($i = n$), the algorithm returns the maximum of A.

Proof. We use induction on i.

Base case ($i = 1$): $max = A[1]$, which is the maximum of $\{A[1]\}$.

Inductive step: assume that after iteration i we have $max = \max\{A[1], \ldots, A[i]\}$. In iteration $i + 1$,

$$A = \big[\underbrace{A[1], A[2], \ldots, A[i]}_{\text{maximum}=max}, A[i+1], \ldots, A[n]\big].$$

If $A[i+1] > max$, then $A[i+1]$ becomes the new maximum; otherwise we keep the previous one. In both cases $max = \max\{A[1], \ldots, A[i+1]\}$. □

2.3 Efficiency: counting steps

How long an algorithm takes depends on many things: the program, but also the input, the programming language, the software, the operating system and the hardware. To compare algorithms, we ignore all of these and look only at the algorithm itself: we count how many elementary steps it performs.

Definition 2.1: Running time

The *running time* $T(n)$ of an algorithm is the number of steps it requires on an input of size n. Unless stated otherwise, we consider the *worst case*: the largest number of steps over all inputs of size n.

Example 3: Counting the steps of Max-Find

```
max ← A[1]                                   ▷ 1 step
for i = 1 to n do                            ▷ n steps
    if max < A[i] then                       ▷ n steps
        max ← A[i]                           ▷ k ≤ n steps
    end if
end for
return max                                   ▷ 1 step
```

Hence $T(n) = 2n + k + 2$. The number k of updates depends on the input:

- *Best case*: the maximum is $A[1]$, so the update never runs ($k = 0$).
- *Worst case*: the array is sorted in increasing order, so the update runs every time ($k = n$).

In the worst case,

$$T(n) = 2n + n + 2 = 3n + 2.$$

The same algorithm can be written with a `while` loop:

Example 4: Max-Find with a while loop

```
max ← A[1]                                   ▷ 1 step
i ← 1
while i ≤ n do                               ▷ n steps
    if A[i] > max then                       ▷ n steps
        max ← A[i]                           ▷ ≤ n steps
    end if
    i ← i + 1                                ▷ n steps
end while
return max                                   ▷ 1 step
```

In the worst case $T(n) = 4n + 2$.

Both versions are correct and do the same thing, and both running times are linear functions of n. The exact constants depend on how we count: initializing i is one more step, and the loop test runs once more when it finally fails. Such details change only the constants, which is why we will learn to ignore them.

Start of lecture 29 September

2.4 Asymptotic notation

In the previous section we counted the steps of two versions of Max-Find, one with a `for` loop and one with a `while` loop, and obtained $T(n) = 3n + 2$ and $T(n) = 4n + 2$. Which

one is faster? The exact numbers depend on how we count, so comparing them directly tells us little. What matters is how the running time changes as the input grows: if n doubles, both running times roughly double. In this sense, both *grow like* n.

Asymptotic ("big O") notation makes this idea precise. It ignores constant factors and lower-order terms: the $+2$ can be dropped, and $4n$ grows faster than $3n$ only by a constant factor, which makes no real difference.

Definition 2.2: O, Ω and Θ

Let $T, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$.

- $T(n) = O(g(n))$ if there are constants $c > 0$ and n_0 such that $T(n) \leq c\, g(n)$ for all $n \geq n_0$: g is an *upper bound* on the growth of T.
- $T(n) = \Omega(g(n))$ if there are constants $c > 0$ and n_0 such that $T(n) \geq c\, g(n)$ for all $n \geq n_0$: g is a *lower bound*.
- $T(n) = \Theta(g(n))$ if both $T(n) = O(g(n))$ and $T(n) = \Omega(g(n))$: T grows exactly like g, up to constants.

Informally, O means "at most", Ω means "at least" and Θ means "exactly", always up to constant factors and for large n. The running time of Max-Find is $T(n) = \Theta(n)$: it grows like a linear function of n.

Proposition 2.2: Combining upper and lower bounds

$$T(n) = O(g(n)) \text{ and } T(n) = \Omega(g(n)) \iff T(n) = \Theta(g(n)).$$

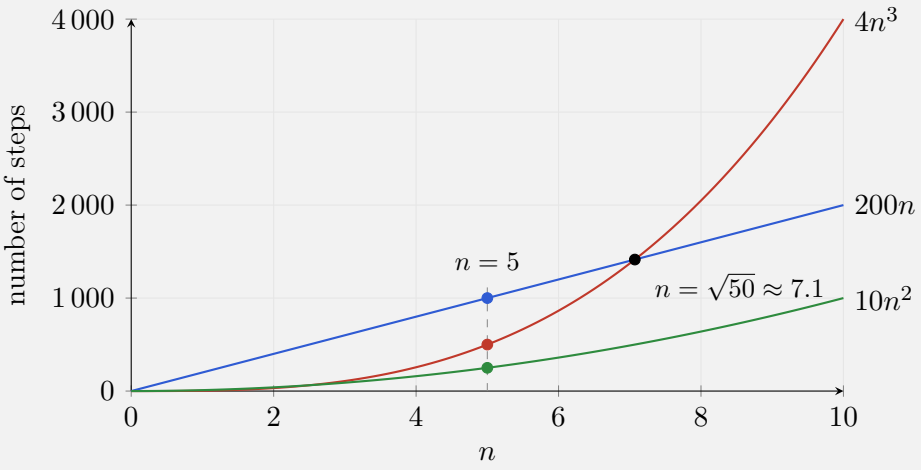
This is the asymptotic version of "$x \leq 3$ and $x \geq 3$ imply $x = 3$". For instance, $T(n) = O(n^3)$ and $T(n) = \Omega(n^3)$ give $T(n) = \Theta(n^3)$.

Upper bounds are not unique: if $T(n) = \Theta(n^2)$, then also $T(n) = O(n^2)$, $T(n) = O(n^3)$, $T(n) = O(n^3 \log n)$, and so on.

Example 5: The dominant term

Let $T(n) = 4n^3 + 10n^2 + 200n + 7$. How fast does this function grow?

The plot compares the three terms that depend on n.



For small n, a lower-order term can be the largest. At $n = 5$ we have

$$200n = 1000, \qquad 4n^3 = 500, \qquad 10n^2 = 250.$$

As n grows, however, the cubic term overtakes $200n$ at $n = \sqrt{50} \approx 7.1$ and dominates from then on, so it determines the running time.

Upper bound. For $n \geq 1$, every power of n in $T(n)$ is at most n^3, so

$$T(n) \leq 4n^3 + 10n^3 + 200n^3 + 7n^3 = 221\, n^3.$$

Hence $T(n) = O(n^3)$, with $c = 221$ and $n_0 = 1$.

Lower bound. All the terms are non-negative, so

$$T(n) \geq 4n^3.$$

Hence $T(n) = \Omega(n^3)$, with $c = 4$ and $n_0 = 1$.

Conclusion. Therefore, by Proposition 2.2, $T(n) = \Theta(n^3)$.

Other upper bounds. For $n \geq 1$,

$$T(n) \leq 221\, n^3 \leq 221\, n^4 \leq 221\, n^5,$$

so we also have $T(n) = O(n^4)$ and $T(n) = O(n^5)$. However, $T(n) \neq \Theta(n^4)$: since $T(n)/n^4 \leq 221/n \to 0$, no constant $c > 0$ gives $T(n) \geq c\, n^4$ for all large n.

When analysing an algorithm we are mostly interested in the *upper bound*, that is, in the worst case. Often we do not know exactly how many times a step runs. In Algorithm 2.1, we do not know in advance how many times the update $max \leftarrow A[i]$ will run, but in the worst case (a sorted array) it runs n times, so it never runs more than n times. Bounding each step from above gives an upper bound on $T(n)$.

2.4.1 Comparing growth rates

Proposition 2.3: Growth rates

For large n (that is, up to constants and eventually):

- $n^c \leq n^d$ for $c \leq d$;
- $\log^c n = (\log n)^c \leq n^d$ for all constants $c, d > 0$;
- $n^c \leq d^n$ for every constant c and every $d > 1$;
- more generally, $n^c \leq d^{an^b}$ for $d > 1$ and constants $a, b > 0$.

Consequently,

$$\begin{aligned} 1 \leq \log\log n \leq \log n \leq \log^2 n \leq \sqrt{n} \leq n \\ \leq n \log\log n \leq n \log n \leq n\sqrt{n} \leq n^2 \leq n^3 \leq 2^n. \end{aligned}$$

For example, $(\log n)^{10}$ and $\sqrt{n}$: the logarithm raised to any power always loses eventually, so $\sqrt{n}$ grows faster.

Note: The base of the logarithm does not matter

Changing the base of a logarithm only multiplies it by a constant:

$$\log_2 n = \frac{\log_{10} n}{\log_{10} 2} = \underbrace{\frac{1}{\log_{10} 2}}_{\text{constant}} \cdot \log_{10} n.$$

Asymptotically all bases are equivalent, so we simply write $O(\log n)$.

An exponential running time means that the algorithm is extremely slow: an algorithm with exponential running time is hopeless except for tiny inputs. Ideally we would like logarithmic or linear running times.

n	$\log_2 n$	$n \log_2 n$	n^2	2^n
10	3.3	33	10^2	$\approx 1.0 \cdot 10^3$
100	6.6	664	10^4	$\approx 1.3 \cdot 10^{30}$
1000	10.0	9966	10^6	$\approx 1.1 \cdot 10^{301}$

2.5 Lower bounds and optimal algorithms

Max-Find is correct and runs in time $\Theta(n)$. Could we do better, say $O(\sqrt{n})$? No: any correct algorithm must read every element, because an element it never reads could be the maximum. With n elements to read, no algorithm can beat $\Omega(n)$.

Definition 2.3: Lower bound and optimal algorithm

A problem has *lower bound* $\Omega(g(n))$ if every algorithm that solves it needs $\Omega(g(n))$ steps in the worst case. An algorithm is (asymptotically) *optimal* if its running time matches a lower bound for the problem.

Example 6: Max-Find is optimal

Max-Find (Algorithm 2.1) runs in $O(n)$, as we computed in Example 3, and the problem requires $\Omega(n)$, so Max-Find is *optimal*.

2.6 Prefix sums

Problem: Prefix sums

Input: an array A of length n.
Output: an array S of length n such that, for every i,

$$S[i] = \sum_{j=1}^{i} A[j].$$

For example, $A = [3, 6, -2, -5, 8]$ gives $S = [3, 9, 7, 2, 10]$.

2.6.1 A quadratic algorithm

The direct approach computes each sum from scratch.

Algorithm 2.2: Prefix sums, direct version

```
1: n ← length(A)                                  ▷ 1 step
2: for i = 1 to n do                              ▷ n steps
3:     S[i] ← 0                                   ▷ n steps
4:     for j = 1 to i do                          ▷ i times for the i-th element
5:         S[i] ← S[i] + A[j]                     ▷ i times
6:     end for
7: end for
8: return S                                       ▷ 1 step
```

The inner loop runs once for the first element, twice for the second, and n times for the last. Each of its two lines therefore runs

$$1 + 2 + 3 + \cdots + n = \sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

times, and

$$T(n) = 1 + n + n + 2 \cdot \frac{n(n+1)}{2} + 1 = n^2 + 3n + 2 = \Theta(n^2).$$

Note: A quicker upper bound

We do not need the exact sum. For every i, the inner loop runs at most n times, so

$$T(n) \leq 2n + n \cdot (2n) + 2 = O(n^2).$$

Bounding each loop by its largest possible number of iterations is often enough to obtain an upper bound.

2.6.2 Closing the gap

Is this algorithm optimal? No: the lower bound for the problem is $\Omega(n)$ (we must read every element), while our algorithm takes $\Theta(n^2)$. There is a *gap*, and we can try to close it by finding a better algorithm. The key observation is that each prefix sum extends the previous one:

$$S[i] = \sum_{j=1}^{i} A[j] = \sum_{j=1}^{i-1} A[j] + A[i] = S[i-1] + A[i].$$

Algorithm 2.3: Prefix sums, fast version

```
1: n ← length(A)
2: S[1] ← A[1]
3: for i = 2 to n do
4:     S[i] ← S[i − 1] + A[i]                    ▷ n − 1 times
5: end for
6: return S
```

Now $T(n) = \Theta(n)$, which matches the lower bound: the fast version is optimal.

2.7 An open gap: matrix multiplication

Let $A, B \in \mathbb{R}^{n \times n}$ and $C = AB$. Each entry is

$$C[i,j] = \sum_{k=1}^{n} A[i,k]\, B[k,j].$$

Computing one entry takes n multiplications, and there are n^2 entries, so the direct algorithm runs in $\Theta(n^3)$.

A lower bound is $\Omega(n^2)$: the output matrix C has n^2 values, and we must compute each of them. For many years, researchers have tried to close the gap between n^2 and n^3. The gap has been reduced, but it has not yet been closed.

Note: For reference

Strassen's algorithm (1969) multiplies two $n \times n$ matrices in $O(n^{\log_2 7}) \approx O(n^{2.81})$ operations. The best known exponent today is about 2.37, still above the lower bound 2.

3 Recursion

3.1 Fibonacci numbers

Definition 3.1: Fibonacci sequence

$$\text{fib}(n) = \begin{cases} 1, & n = 0 \text{ or } n = 1, \\ \text{fib}(n-1) + \text{fib}(n-2), & n \geq 2. \end{cases}$$

n	0	1	2	3	4	5	6	7	8	9
$\text{fib}(n)$	1	1	2	3	5	8	13	21	34	55

The definition is *recursive*: it defines $\text{fib}(n)$ in terms of smaller values. It translates directly into a recursive algorithm, one that calls itself.

Algorithm 3.1: Recursive Fibonacci

```
1: function FIBONACCI(n)
2:     if n = 0 or n = 1 then                          ▷ 1 step
3:         return 1                                    ▷ at most 1 step
4:     end if
5:     f1 ← FIBONACCI(n − 1)                           ▷ T(n − 1) steps
6:     f2 ← FIBONACCI(n − 2)                           ▷ T(n − 2) steps
7:     return f1 + f2                                  ▷ 1 step
8: end function
```

3.1.1 Running time of the recursive algorithm

Let $T(n)$ be the number of steps for input n. Counting the steps of Algorithm 3.1 gives a *recurrence*:

$$T(n) = \begin{cases} 2, & n = 0 \text{ or } n = 1, \\ T(n-1) + T(n-2) + 3, & n \geq 2. \end{cases}$$

The formula is recursive. How can we solve it? A first idea is to expand it. The recurrence holds for *every* $n \geq 2$, so we can apply it to the terms on the right as well: replacing n with $n-1$ gives $T(n-1) = T(n-2) + T(n-3) + 3$, and replacing n with $n-2$ gives $T(n-2) = T(n-3) + T(n-4) + 3$. In the computations below, each underlined term is replaced, in the next line, by the expression underlined in the same colour.

First expansion.

$$\begin{aligned} T(n) &= \underline{T(n-1)} + \underline{T(n-2)} + 3 \\ &= \underline{\big[T(n-2) + T(n-3) + 3\big]} + \underline{\big[T(n-3) + T(n-4) + 3\big]} + 3 \\ &= T(n-2) + 2T(n-3) + T(n-4) + 9. \end{aligned}$$

Second expansion. We expand each of the three terms again:

$$\begin{aligned} T(n) &= \underline{T(n-2)} + \underline{2T(n-3)} + \underline{T(n-4)} + 9 \\ &= \underline{\big[T(n-3) + T(n-4) + 3\big]} + \underline{2\big[T(n-4) + T(n-5) + 3\big]} \\ &\quad + \underline{\big[T(n-5) + T(n-6) + 3\big]} + 9 \\ &= T(n-3) + 3T(n-4) + 3T(n-5) + T(n-6) + 21 = \cdots \end{aligned}$$

The expansion stops when the arguments reach 0 or 1. After two expansions the smallest argument is already $n-6$, and the number of terms keeps growing: we may suspect that $T(n)$ is exponential.

Proposition 3.1: The recursive algorithm is exponential

The running time of Algorithm 3.1 satisfies

$$T(n) = \Omega\big(2^{n/2}\big) = \Omega\big((\sqrt{2})^n\big).$$

Proof. T is non-decreasing: $T(1) = T(0)$, and for $n \geq 2$ we have $T(n) \geq T(n-1)$.

Replacing $T(n-1)$ with the smaller $T(n-2)$,

$$T(n) = T(n-1) + T(n-2) + 3 \geq 2\,T(n-2) + 3.$$

Applying the same inequality repeatedly,

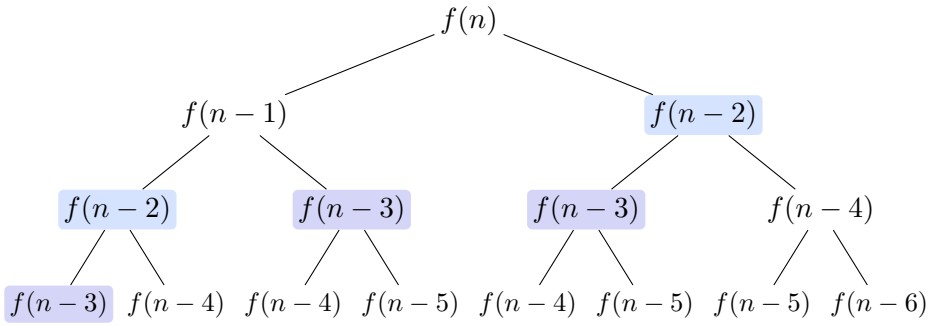
$$\begin{aligned}
T(n) &\geq 2\,\big(2T(n-4)+3\big) + 3 = 4\,T(n-4) + 9 \\
&\geq 4\,\big(2T(n-6)+3\big) + 9 = 8\,T(n-6) + 21 \\
&\geq 8\,\big(2T(n-8)+3\big) + 21 = 16T(n-8) + 45 \\
&\;\;\vdots \\
&\geq 2^i T(n-2i) + 3(2^i - 1) \geq 2^i T(n-2i) \qquad \text{after } i \text{ steps.}
\end{aligned}$$

For even n, take $i = n/2$:

$$T(n) \geq 2^{n/2}\,T(0) = 2 \cdot 2^{n/2} = 2(\sqrt{2})^n.$$

For odd n, take $i = (n-1)/2$ and use $T(1) = 2$ in the same way. □

Why is it so slow? To compute fib(n), the algorithm makes two calls, fib$(n-1)$ and fib$(n-2)$, each of which makes two more calls, and so on. It computes the same values a lot of times, again and again: the work roughly doubles at each level of the recursion tree.



Here $f(k)$ stands for a call to fib(k). Already in the first levels, $f(n-2)$ is computed twice and $f(n-3)$ three times.

3.1.2 Avoiding repeated work

There are two ways to fix the problem:

- **Store the values** (memoization): using some memory, we compute each fib(k) once and look it up afterwards.
- **Compute bottom-up**: start from the base cases and fill the table from left to right, $1+1=2$, $2+1=3$, $3+2=5$, $5+3=8$, …, each value being the sum of the previous two.

Algorithm 3.2: Iterative Fibonacci

```
1: function FibonacciIterative(n)
2:     if n ≤ 1 then
3:         return 1
4:     end if
5:     previous ← 1                                   ▷ fib(0)
6:     current ← 1                                    ▷ fib(1)
7:     for i = 2 to n do
8:         new ← previous + current
9:         previous ← current
10:        current ← new
11:    end for
12:    return current
13: end function
```

The loop runs $n-1$ times with a constant amount of work per iteration, so the running time is $T(n) = O(n)$: this algorithm is linear.

Note: Indexing and closed form

The slides use the convention $F_0 = 0$, $F_1 = 1$ (their iterative algorithm returns n for $n \leq 1$ and starts from 0 and 1). With that convention $\text{fib}(n) = F_{n+1}$, so the two sequences differ only by a shift. There is also a closed formula, which we do not prove:

$$F_n = \frac{1}{\sqrt{5}}\left[\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n\right], \qquad \text{fib}(n) = F_{n+1}.$$

Start of lecture 1 October

3.2 Binary search

Problem: Search

Input: an array A of n numbers and a value x.
Output: TRUE if x is in A, FALSE otherwise.

We could compare x with every element, with linear complexity. If A is *sorted*, we do not need to check all the elements: we look at the central element and compare it with x. If x is larger, it can only be in the right half; if smaller, only in the left half. We then repeat on that half.

Example 7: Searching a sorted array

Let $A = [1, 3, 7, 10, 13, 24, 30, 33, 39, 55]$ and $x = 39$.

1. The central element is $A[5] = 13 < 39$, so we keep the right part $[24, 30, 33, 39, 55]$.
2. Its central element is $33 < 39$, so we keep $[39, 55]$.
3. Its central element is $39 = x$: found.

Algorithm 3.3: Binary search

```
1: function BinarySearch(A, x)
2:     n ← len(A)
3:     if n = 0 then
4:         return FALSE
5:     end if
6:     if x < A[1] or x > A[n] then                ▷ x outside the range of A
7:         return FALSE
8:     end if
9:     mid ← ⌈n/2⌉
10:    if x = A[mid] then
11:        return TRUE
12:    else if x < A[mid] then
13:        return BinarySearch(A[1, ..., mid − 1], x)
14:    else
15:        return BinarySearch(A[mid + 1, ..., n], x)
16:    end if
17: end function
```

The *ceiling* $\lceil y \rceil$ is the smallest integer greater than or equal to y: for example, $\lceil 3 \rceil = 3$ and $\lceil 3.1 \rceil = 4$.

Example 8: An unsuccessful search

Let $A = [13, 17]$ and $x = 15$.

1. Since $13 < 15 < 17$, x lies within the range of A.
2. The central element is $A[1] = 13 < 15$ (here $mid = \lceil 2/2 \rceil = 1$), so we keep the right part $[17]$.
3. Now $15 < 17$, so x is outside the range of $[17]$: the algorithm returns FALSE.

3.2.1 Running time

Every line of Algorithm 3.3 takes constant time, except the recursive call. The call works on an array of half the size, so it takes $T(n/2)$ steps instead of $T(n)$:

$$T(n) = \begin{cases} c_1, & n \leq 1, \\ T(n/2) + c_2, & \text{otherwise}, \end{cases}$$

where c_1 and c_2 are constants. For $n \geq 2$,

$$T(n) = T\left(\tfrac{n}{2}\right) + c_2 = T\left(\tfrac{n}{4}\right) + 2c_2 = T\left(\tfrac{n}{8}\right) + 3c_2 = \cdots = T\left(\tfrac{n}{2^i}\right) + i\,c_2.$$

We stop when the array has size 1, that is, when $n/2^i = 1$, or $i = \log_2 n$. Then $T(n/2^i) = T(1) = c_1$ and

$$T(n) = c_1 + c_2 \log_2 n = \Theta(\log n).$$

Note: Subarrays

We treat $A[1, \ldots, mid - 1]$ as a view of the original array, described by its first and last index. Copying the subarray at every call would cost linear time and destroy the logarithmic bound.

We cannot do better than $\Theta(\log n)$, but we are not able to prove this yet. Splitting into three parts instead of two does not help asymptotically: the recurrence becomes $T(n) = T(n/3) + c_2'$, so $T(n) = O(\log_3 n)$, and $O(\log_3 n) = O(\log_2 n)$ because $\log_3 n = \log_2 n / \log_2 3$.

4

Complexity classes

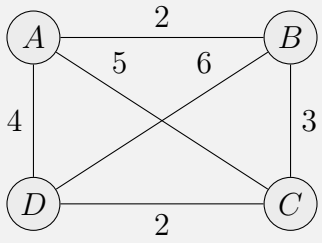
4.1 The traveling salesman problem

We want to visit all the nodes of a weighted graph, starting from a node and coming back to it, while minimizing the total cost. Every order of visit gives a tour, and different tours have different costs.

Problem: Traveling salesman problem (TSP)

Input: n cities and the cost of travelling between each pair of them.
Output: a tour that visits every city exactly once and returns to the starting city, with minimum total cost.

Example 9: Four cities



Starting from A, there are $3! = 6$ orders of visit, which give three distinct tours (each one can be travelled in two directions):

$$A\,B\,C\,D\,A: \ 2+3+2+4=11,$$
$$A\,B\,D\,C\,A: \ 2+6+2+5=15,$$
$$A\,C\,B\,D\,A: \ 5+3+6+4=18.$$

The optimal tour costs 11. In the lecture graph with eight cities, two tours cost 15 and 21.

The brute-force approach tries all tours: with n cities there are $(n-1)!$ orders of visit, and

$$n! = 1 \cdot 2 \cdot 3 \cdots (n-1) \cdot n \geq 1 \cdot 2 \cdot 2 \cdots 2 = 2^{n-1},$$

so the factorial grows even faster than an exponential: this is not good at all. No known algorithm finds the shortest tour quickly.

4.2 Decision problems, P and NP

We usually study *decision problems*, whose answer is yes or no. An optimization problem such as minimizing costs or maximizing profits becomes a decision problem by fixing a threshold. For TSP: *is there a tour that visits all the nodes with total cost at most* 14*?* If the answer is yes, we can ask whether we can do better, for example cost at most 13.

Definition 4.1: The class P

P is the class of problems that we can solve in *polynomial time* (poly-time), that is, in time $O(n^k)$ for some constant k.

Definition 4.2: The class NP

NP is the class of problems for which, if someone gives us a candidate solution, we can verify in polynomial time whether the answer is YES.

Example 10: Easy to verify

- *Rubik's cube*: it is easy to check whether a cube is solved.
- *TSP*: given a tour, we add up its costs and check whether the total is at most 14.
- *Sudoku*: it is easy to check whether a filled grid is correct.

What these problems have in common is that, given a solution, we can verify in poly-time whether it is correct, even though finding one may be hard.

Max-Find and binary search are in P. Every problem in P is also in NP: if we can solve a problem quickly, we can verify an answer by solving it ourselves. Hence $\mathrm{P} \subseteq \mathrm{NP}$.

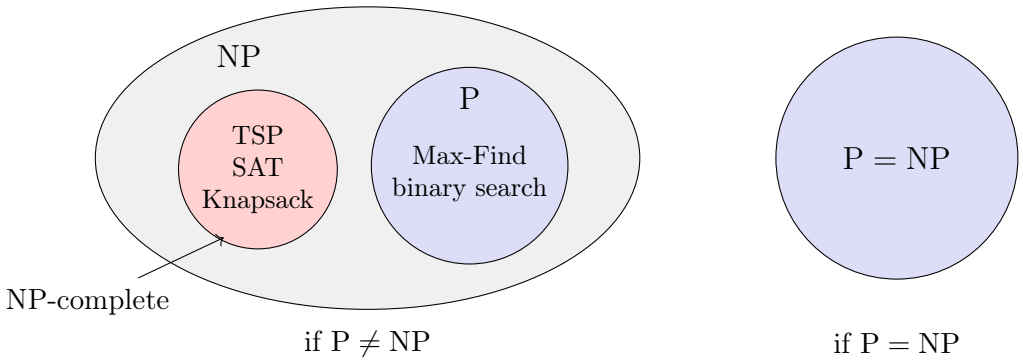
Note: What NP stands for

NP stands for *nondeterministic polynomial time*, not "non polynomial". A problem in NP may well be easy: every problem in P is in NP.

Are there problems in NP that are not in P? Nobody knows.

$$\textit{Is } \mathrm{P} = \mathrm{NP}\textit{, or is } \mathrm{P} \subsetneq \mathrm{NP}\textit{? We don't know.}$$

This is one of the most important open questions in computer science.



4.3 Reductions and NP-completeness

Some problems in NP are as hard as any other problem in NP.

Definition 4.3: NP-complete problem

A problem X is *NP-complete* if:

1. $X \in$ NP;
2. every problem $Y \in$ NP can be *reduced* in polynomial time to X.

A reduction of Y to X solves Y using an algorithm for X plus a polynomial amount of extra work.

Example 11: Max-Find and sorting

If we can sort an array in poly-time, we can also find its maximum in poly-time: sort it and take the last element. Max-Find reduces to sorting.

The converse also holds: if we can find the maximum, we put it in the last position and repeat on the remaining elements, until the whole array is sorted. This uses n calls to Max-Find, a polynomial amount of work, so sorting reduces to Max-Find.

Surprisingly, NP-complete problems exist, such as *SAT* (Boolean satisfiability), which we will not study in detail. If P = NP, the classes collapse: NP-complete problems would be solvable in polynomial time, like every other problem in NP.

Note: Are TSP, Sudoku and Rubik's cube NP-complete?

Complexity classes describe how the cost grows with the input size, so we must consider generalized versions of these puzzles:

- the decision version of TSP ("is there a tour of cost at most k?") is NP-complete;

- Sudoku on $n^2 \times n^2$ grids ("can this partially filled grid be completed?") is NP-complete;
- for the $n \times n \times n$ Rubik's cube, deciding whether a position can be solved in at most k moves is NP-complete (proved in 2018).

The classic 9×9 Sudoku and $3 \times 3 \times 3$ cube have a fixed size, so asymptotic statements do not apply to them directly.

Start of lecture 5 October

Corollary 4.1: Solving one NP-complete problem

Let X be NP-complete. If X can be solved in poly-time, then every problem in NP can be solved in poly-time. Equivalently, if some $Y \in \text{NP}$ cannot be solved in poly-time, then neither can X.

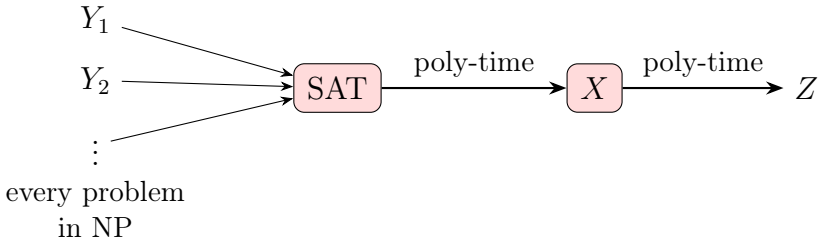
SAT was the first problem proved to be NP-complete: every problem in NP can be reduced to SAT (the Cook–Levin theorem). To show that a new problem is NP-complete, we no longer need to reduce every problem in NP to it.

Proposition 4.1: Proving NP-completeness

To prove that a problem X is NP-complete, show that:

1. $X \in \text{NP}$;
2. there is a problem Y already known to be NP-complete and a poly-time reduction of Y to X.

This works because reductions compose: every problem in NP reduces to Y, and Y reduces to X, so every problem in NP reduces to X. The composition of two polynomials is still a polynomial.



4.4 Some NP-complete problems

4.4.1 Knapsack

We want to fill a backpack with the most valuable set of items without exceeding its capacity.

Problem: Knapsack

Input: a set of items $I = \{1, 2, \ldots, n\}$, where item i has a weight w_i and a value v_i, and a budget B.
Output: a subset of items $S \subseteq I$ such that

$$\sum_{i \in S} w_i \leq B \qquad \text{that maximizes} \qquad \sum_{i \in S} v_i.$$

Example 12: A knapsack instance

Let $B = 55$ and

i	1	2	3	4	5	6	7
w_i	10	20	6	30	15	20	50
v_i	10	15	8	17	12	17	20

We must maximize $\sum_{i \in S} v_i$ subject to the constraint $\sum_{i \in S} w_i \leq 55$. One feasible solution is $S = \{1, 2, 3, 5\}$, with total weight 51 and total value 45. A solution taken "at random" like this one is probably not the best: in Example 16 we will find one of value 47.

4.4.2 Set cover

Suppose we want to rob a casino: we need people who can do different things.

Problem: Set cover

Input: a task that requires the skills $s_1, \ldots, s_n$, and m experts $E_1, \ldots, E_m$, where each $E_i \subseteq \{1, \ldots, n\}$ is the subset of skills that expert i has.
Output: the smallest number of experts that together cover all the skills.

In general, the skills are elements $e_1, \ldots, e_n$ and the experts are sets $S_1, \ldots, S_m$ of elements. We want the minimum number of sets whose union contains all the elements.

Example 13: Covering six skills

	s_1	s_2	s_3	s_4	s_5	s_6
E_1	×		×	×		
E_2	×	×				
E_3		×			×	
E_4	×			×		×
E_5			×			×

We must take E_3, because it is the only expert with skill s_5. E_3 covers s_2 and s_5; no single other expert covers all of s_1, s_3, s_4, s_6, so we need at least 3 experts. One optimal solution is $\{E_3, E_4, E_5\}$.

A heuristic method: take E_3, then repeatedly take the expert who covers the largest number of skills not yet covered. Such solutions are not optimal in general.

4.4.3 Subset sum

Problem: Subset sum

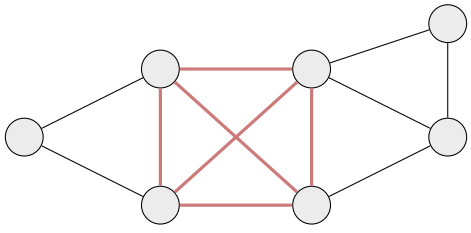
Input: a list of integers.
Output: a nonempty sublist whose elements sum to 0, if one exists.

For the list $[3, 7, -3, -8, 10, -16, 2, 9]$, the sublists $[3, -3]$ and $[7, 9, -16]$ both sum to 0.

4.4.4 Maximum clique

Problem: Maximum clique

Input: a graph.
Output: a largest *clique*, that is, a largest set of nodes that are all connected to one another.



The thick edges form a clique of size 4, the largest in this graph.

Proposition 4.2: Some NP-complete problems

The decision versions of Knapsack, Set cover, Subset sum and Max clique are NP-complete.

We do not prove this: each proof is a poly-time reduction from a known NP-complete problem (Proposition 4.1).

4.5 NP-hard problems

Definition 4.4: NP-hard problem

A problem X is *NP-hard* if every problem in NP can be reduced to X in polynomial time. Unlike an NP-complete problem, X does not have to belong to NP.

NP-hard problems are more general: NP-complete means NP-hard and in NP. For example, the optimization versions of Knapsack and TSP (find the best solution, not just answer yes or no) are NP-hard.

Note: Minimum spanning tree

The minimum spanning tree problem is *not* NP-hard (unless P = NP): Kruskal's and Prim's algorithms solve it in polynomial time. Its relative, the Steiner tree problem, which must connect only a given subset of the nodes, is NP-hard.

5 Approximation and heuristics

For NP-complete and NP-hard problems we do not know fast exact algorithms. Two ways out are approximation algorithms, which come with a guarantee, and heuristics, which do not.

5.1 Approximation algorithms

Definition 5.1: c-approximation algorithm

Let $c \geq 1$. For an input I, let OPT_I be the value of an optimal solution. A *c-approximation algorithm* returns, for every input I, a solution whose value is

- at most $c \cdot \mathrm{OPT}_I$, for a *minimization* problem;
- at least OPT_I / c, for a *maximization* problem.

Example 14: Reading the guarantee

- *TSP (minimization)*: suppose that for an input I the optimal tour costs $\mathrm{OPT}_I = 5000$. A 2-approximation algorithm guarantees a tour of cost at most 10 000: it may cost more than 5000, but we are sure that it does not exceed 10 000. For $c = 1.1$, the tour costs at most $1.1 \cdot 5000 = 5500$, no more than 10% above the optimum.
- *Knapsack (maximization)*: if $\mathrm{OPT}_I = 1000$, a 1.1-approximation guarantees a value of at least $1000/1.1 \approx 909$.

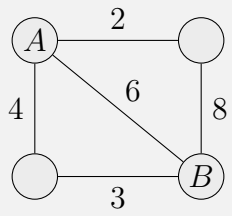
5.1.1 Guarantees without knowing the optimum

How can we prove that a solution is a c-approximation if we cannot find the optimal solution? We may not find OPT_I, but we can often find a *lower bound* $L \leq \mathrm{OPT}_I$ (for a

minimization problem). Then

$$\frac{\text{output}}{\text{OPT}_I} \leq \frac{\text{output}}{L}.$$

Example 15: Using a lower bound



Suppose every solution must travel from A to B. Every route from A to B costs at least 6, so $\text{OPT}_I \geq 6$: 6 is a lower bound. If our algorithm outputs a solution of cost 17, is it good?

$$\frac{\text{output}}{\text{OPT}_I} = \frac{17}{\text{OPT}_I} \leq \frac{17}{6} \approx 2.83,$$

so our solution is at most 17/6 times the optimum.

Start of lecture 8 October

5.1.2 How well can we approximate?

Different problems behave very differently:

- For *set cover*, where we minimize the number of sets, a simple greedy algorithm is, at worst, an $O(\log n)$-approximation.
- Many other problems cannot be approximated well in poly-time. For *max clique*, no polynomial-time algorithm can guarantee even a factor like $\sqrt{n}$, unless P = NP.

Some approximation algorithms are also very inefficient. This is why there is another approach: heuristics.

5.2 Heuristics

A *heuristic* is a method that usually finds good solutions quickly, but without any guarantee on their quality.

5.2.1 A greedy heuristic for knapsack

For each item, consider the value per unit of weight

$$r_i = \frac{v_i}{w_i}.$$

We start putting into the backpack the items with the largest r_i, and keep adding items as long as the constraint is satisfied. This usually gives a good solution, but not always the optimal one.

Algorithm 5.1: Greedy knapsack

sort the items so that $r_{i_1} \geq r_{i_2} \geq \cdots \geq r_{i_n}$
$S \leftarrow \varnothing$, $W \leftarrow 0$
for $k = 1$ **to** n **do**
    if $W + w_{i_k} \leq B$ **then**
        $S \leftarrow S \cup \{i_k\}$, $W \leftarrow W + w_{i_k}$
    end if
end for
return S

Example 16: Greedy on the knapsack instance

For the instance in Example 12,

$$r_1 = 1, \quad r_2 = \tfrac{3}{4}, \quad r_3 = \tfrac{4}{3}, \quad r_4 = \tfrac{17}{30}, \quad r_5 = \tfrac{4}{5}, \quad r_6 = \tfrac{17}{20}, \quad r_7 = \tfrac{2}{5}.$$

Item 3 has the largest ratio, so it is the first one we add. The order is $3, 1, 6, 5, 2, 4, 7$:

- add 3 (weight 6), 1 (weight 16), 6 (weight 36) and 5 (weight 51);
- items 2, 4 and 7 no longer fit ($51 + 20 > 55$).

The result is $S = \{1, 3, 5, 6\}$, with weight 51 and value 47. Trying all subsets shows that 47 is optimal for this instance.

The greedy solution is not optimal in general.

Example 17: When greedy fails

Let $B = 100$, ten items with $w_1 = \cdots = w_{10} = 50$ and $v_1 = \cdots = v_{10} = 50$ (each with ratio 1), and an item 0 with $w_0 = 51$ and $v_0 = 52$, whose ratio is slightly larger than 1. Greedy takes item 0 first, and then no other item fits ($51 + 50 > 100$): the value is 52. Taking two items of weight 50 gives value 100.

No known algorithm solves knapsack in poly-time: it is NP-complete. If we could "break" the items and take a fraction of one of them (the *fractional* knapsack), the greedy algorithm would be perfect: we fill the remaining capacity with a fraction of the next item.

5.2.2 Greedy algorithms and meta-heuristics

Definition 5.2: Greedy algorithm

A *greedy* algorithm makes, at each step, the choice that is best for now, hoping that it will also work well in the future, and continues in the same way.

The knapsack heuristic above and the set cover heuristic are both greedy algorithms. Greediness is an example of a *meta-heuristic*: a general strategy, a wider class that includes many heuristics. Another example is *local search*: starting from a solution, such as a tour, we try small modifications (for example, exchanging two of its edges for two others) and keep those that improve it, searching for the optimal one.

6

Distance measures

6.1 Why distances?

Many problems ask us to find distances or similarities between objects. For example, given some documents and a query, we want the documents most similar to the query; or we want groups of similar documents, a *cluster*. The input is a set of objects and the output a group of them. To do this we use different kinds of distances: a function that measures the distance between two elements defines a relation between them.

6.2 Distance functions

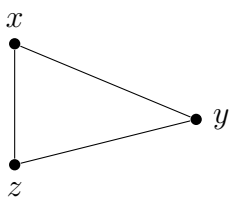
Definition 6.1: Distance function

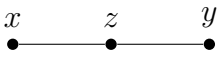
Let U be a universe of elements (documents, objects, strings, ...). A *distance function* is a map

$$d : U \times U \longrightarrow \mathbb{R},$$

taking two elements and returning a number, which satisfies, for all $x, y, z \in U$:

1. $d(x, y) \geq 0$, and $d(x, y) = 0$ if and only if $x = y$;
2. $d(x, y) = d(y, x)$ (symmetry);
3. $d(x, y) \leq d(x, z) + d(z, y)$ (triangle inequality).





equality when z lies between

We should define distances that make sense for the problem: what does it mean for two objects to be "close"? It depends on the problem we are considering.

6.3 L_p distances

We have defined what a distance function is (Definition 6.1). We now look at concrete distances, starting with objects described by vectors of numbers, that is, $U = \mathbb{R}^m$.

Suppose we are designing an app like Google Maps that shows the restaurants near us. On the map, our position and each restaurant are points with two coordinates, that is, vectors in $\mathbb{R}^2$. To decide which restaurants are near, we need a distance between two points. The natural choice is the length of the straight segment that joins them: the Euclidean distance. The same formula works for vectors with any number m of coordinates.

Definition 6.2: Euclidean distance

For $U = \mathbb{R}^m$, $\vec{x} = [x_1, \ldots, x_m]$ and $\vec{y} = [y_1, \ldots, y_m]$, the *Euclidean distance*, or L_2 distance, is

$$d(\vec{x}, \vec{y}) = \sqrt{\sum_{j=1}^{m} (x_j - y_j)^2}.$$

It is clearly non-negative and symmetric; it also satisfies the triangle inequality, but we do not prove it.

Definition 6.3: L_p distance

For a constant $p \geq 1$,

$$d_p(\vec{x}, \vec{y}) = \left(\sum_{j=1}^{m} |x_j - y_j|^p \right)^{1/p} = \|\vec{x} - \vec{y}\|_p.$$

Changing p gives different distances, which weigh the differences between coordinates differently:

- $p = 2$: the Euclidean distance.
- $p = 1$: the *Manhattan distance*, $\sum_j |x_j - y_j|$.
- $p \to \infty$: the L_∞ distance, $d_\infty(\vec{x}, \vec{y}) = \max_{i \in \{1,\ldots,m\}} |x_i - y_i|$.
- "L_0": the number of coordinates in which the vectors differ, $|\{i : x_i \neq y_i\}|$. For binary strings this is the *Hamming distance*.

Example 18: Counting the coordinates that differ

- The binary strings 00110 and 10100 differ in 2 positions (the first and the fourth), so their Hamming distance is 2.
- The vectors $[3, 1, 3]$ and $[0, 1, 3]$ differ only in the first coordinate, so their L_0 distance is 1.

Example 19: Comparing L_p distances

Let

$$\vec{x} = [3, 7, 0, 2, 1], \qquad \vec{y} = [4, 5, 10, 7, 3], \qquad \vec{z} = [5, 2, 5, 6, 5].$$

The coordinate differences are $|\vec{x} - \vec{y}| = [1, 2, 10, 5, 2]$ and $|\vec{x} - \vec{z}| = [2, 5, 5, 4, 4]$.

p	$\|\vec{x} - \vec{y}\|_p$	$\|\vec{x} - \vec{z}\|_p$
1	$1 + 2 + 10 + 5 + 2 = 20$	$2 + 5 + 5 + 4 + 4 = 20$
2	$(1 + 4 + 100 + 25 + 4)^{1/2} \approx 11.58$	$(4 + 25 + 25 + 16 + 16)^{1/2} \approx 9.27$
3	$(1 + 8 + 1000 + 125 + 8)^{1/3} \approx 10.45$	$(8 + 125 + 125 + 64 + 64)^{1/3} \approx 7.28$
∞	10	5

With $p = 1$ the two distances are equal. As p increases, both distances decrease, and the larger differences weigh more: $\vec{y}$ differs from $\vec{x}$ by 10 in one coordinate, so it ends up farther than $\vec{z}$, whose differences are moderate everywhere.

6.4 Distances between sets

So far we have worked with vectors. Now imagine that we want to compare the customers of a supermarket based on what they bought.

Example 20: Shopping baskets

Each letter is a product: the first person bought apples (A), bananas (B), carrots (C) and donuts (D):

$$P_1 = \{A, B, C, D\}, \qquad P_2 = \{A, E, G, H\}, \qquad P_3 = \{B, C, E, F\}.$$

Intuitively, persons 1 and 3 are more similar than persons 1 and 2.

6.4.1 Sets as binary vectors

We can write each set as a binary vector, with one coordinate per product, and use the L_1 distance:

	A	B	C	D	E	F	G	H
P_1	1	1	1	1	0	0	0	0
P_2	1	0	0	0	1	0	1	1
P_3	0	1	1	0	1	1	0	0

Then $|P_1 - P_2| = [0, 1, 1, 1, 1, 0, 1, 1]$ and $|P_1 - P_3| = [1, 0, 0, 1, 1, 1, 0, 0]$, so

$$\|P_1 - P_2\|_1 = 6, \qquad \|P_1 - P_3\|_1 = 4.$$

So P_1 is closer to P_3, as we expected. In general,

$$\|P - Q\|_1 = |P| + |Q| - 2|P \cap Q| = |P \triangle Q|,$$

the size of the *symmetric difference* $P \triangle Q$, the elements in exactly one of the two sets.

6.4.2 Counting common items

The formula above depends on the intersection $P \cap Q$. Let us look at the elements in common directly:

$$|P_1 \cap P_2| = 1, \qquad |P_1 \cap P_3| = 2.$$

Is this a good distance? We should check the three properties. Symmetry certainly holds, but the larger the value, the *closer* the two sets: this is a *similarity*, not a distance. It is often more natural to talk about similarity than about distance.

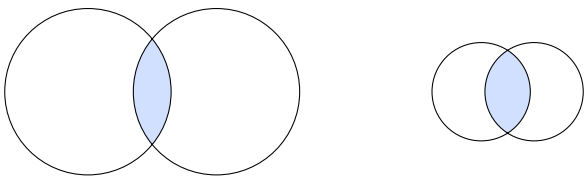
Definition 6.4: Similarity function

A *similarity function* is a map $s : U \times U \to \mathbb{R}$ such that $s(x, y) = s(y, x)$ for all $x, y \in U$, and whose value is larger the more similar x and y are. Similarities are often normalized to take values in $[0, 1]$, with $s(x, x) = 1$ for every x.

Unlike distances, similarities have no standard list of properties: the ones above are the usual conventions. The two notions go in opposite directions: a large similarity means a small distance. Here we do not see a "natural" distance function, and we cannot simply take the negative of the similarity, because distances cannot be negative.

6.4.3 Normalizing the intersection

The size of the intersection alone, however, is misleading. Two pairs of sets can have intersections of the same size, but if the sets in the second pair are smaller, the intersection is a larger fraction of them, so they are more similar.



1 2

Same intersection, different set sizes: the sets in pair 2 are more similar, because their intersection is a larger fraction of them.

We can normalize the intersection in several ways:

$$\frac{|P \cap Q|}{\max\{|P|, |Q|\}}, \qquad \frac{|P \cap Q|}{|P \cup Q|}, \qquad \frac{|P \cap Q|}{|P| + |Q|}.$$

Definition 6.5: Jaccard similarity

The *Jaccard similarity* of two finite sets P and Q is

$$J(P, Q) = \frac{|P \cap Q|}{|P \cup Q|},$$

where $|\cdot|$ is the cardinality (not the absolute value).

The Jaccard similarity is symmetric, lies between 0 and 1, and gives $J(P, P) = 1$ for every nonempty P: it is a normalized similarity function (Definition 6.4). For the baskets in Example 20,

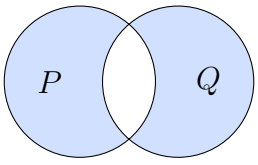
$$J(P_1, P_2) = \frac{|\{A\}|}{|\{A, B, C, D, E, G, H\}|} = \frac{1}{7}, \qquad J(P_1, P_3) = \frac{|\{B, C\}|}{|\{A, B, C, D, E, F\}|} = \frac{2}{6} = \frac{1}{3},$$

so P_1 is more similar to P_3, as expected.

Definition 6.6: Jaccard distance

We turn the Jaccard similarity into a distance by taking

$$d_J(P, Q) = 1 - \frac{|P \cap Q|}{|P \cup Q|} = \frac{|P \cup Q| - |P \cap Q|}{|P \cup Q|} = \frac{|P \triangle Q|}{|P \cup Q|} = \frac{|P \setminus Q| + |Q \setminus P|}{|P \cup Q|}.$$



The symmetric difference $P \triangle Q$: everything except the intersection.

One can prove that the Jaccard distance d_J is a distance in the sense of Definition 6.1. For the baskets, $d_J(P_1, P_2) = 6/7$ and $d_J(P_1, P_3) = 4/6 = 2/3$.

7

Distances between strings

Start of lecture 12 October

7.1 Why compare strings?

Biological data are sequences of bases such as `CTAGGATCGGAC`, and DNA sequences have particular properties: we may want to know how "similar" two parts of DNA are. Search engines need similarity between strings too: if we type "Sceince" instead of "Science" on Google, it still understands what we mean.

Example 21: Which strings are closer?

$$X = \texttt{CTAGGAT}, \qquad Y = \texttt{CCAGAGT}, \qquad Z = \texttt{AGCCCTA}.$$

Which pair is more similar, X and Y, or X and Z? We would say X and Y. The answer is intuitive, but which property are we using? How do we define a "distance measure" between strings?

7.2 First attempts

- **Compare letter by letter.** We could compare the strings position by position, as with the Hamming or L_1 distance. But what if the strings have different lengths?
- **Letters as numbers.** Another idea that might come to mind is to associate each letter with its position in the alphabet ($A = 1$, $C = 3$, $G = 7$, $T = 20$) and to compute the L_1 distance between the resulting vectors. For example,

`CAGT` :	3	1	7	20
`ACTT` :	1	3	20	20
difference :	2	2	13	0

$$\Longrightarrow \qquad 2 + 2 + 13 + 0 = 17.$$

The larger the number, the farther the strings, and the distance is symmetric. However, it does not work well:

  - `SAPIENZA` and `SSAPIENZA`, typed with an extra `S` by mistake, are almost identical, but the extra letter shifts all the following ones and the distance explodes;
  - the distance between A and T is larger than between A and B only because of the alphabetical order.

- **Count the characters.** We could count how many times each character appears. Biologically, however, `AAGGGTT` and `TGAAGTG` are very different, even though they contain exactly the same characters.
- **Look at common pieces.** `CATGAGG` and `CAGAGT` share `CA` and `GAG`, even though they have different lengths.
- **Count modifications.** We could take the second string and count how many changes are needed to make it equal to the first. For `SAPIENZA` and `SAAPIEMZA`, we delete the extra `A` and replace `M` with `N`: two changes, so the distance is 2.

The last two ideas are the promising ones: common pieces lead to subsequences, and modifications lead to the edit distance. Consider sequences over an alphabet.

7.3 Subsequences

Definition 7.1: Subsequence

A *subsequence* of a sequence S is a sequence that contains a subset of the characters of S in the same order, not necessarily consecutive.

Example 22: Subsequences

For $S =$ `ATCGGACTGA`, all of the following are subsequences: `ATCGGACTGA` itself, `ACGGCTA`, `AAA`, and the empty string.

Definition 7.2: Common subsequence

Given two sequences S and T, a *common subsequence* is a sequence that is a subsequence of both S and T.

For $S =$ `ATGCATG` and $T =$ `AGACTAG`, `AAT` and `AGAG` are common subsequences, and so is the empty string.

7.3.1 The longest common subsequence

Definition 7.3: Longest common subsequence (LCS)

The *longest common subsequence* of S and T is a common subsequence of maximum length. We write $\mathrm{LCS}(S, T)$ for its length.

We can define the similarity of two strings as the length of their LCS.

Example 23: Longest common subsequences

- For `ATGA` and `TAG`, both `AG` and `TG` are longest common subsequences: the LCS need not be unique, and here none is longer than 2.
- Compare `SAPEMZA` with `SAPIENZA` and with `SALERNO`:

  $\mathrm{LCS}(\texttt{SAPIENZA}, \texttt{SAPEMZA}) = 6$ (`SAPEZA`), $\mathrm{LCS}(\texttt{SAPEMZA}, \texttt{SALERNO}) = 3$ (`SAE`).

  `SAPEMZA` has more in common with `SAPIENZA` than with `SALERNO`.
- In Example 21, $\mathrm{LCS}(X, Y) = 5$ (for example `CAGGT`) while $\mathrm{LCS}(X, Z) = 3$ (`CTA`), which confirms our intuition.
- For `CAGTGACCGATAC` and `GACCAGTCAGCAC` it is already hard to find the LCS by hand. Its length is 8, for example `CAGTGCAC`.

Checking all subsequences is not an option: a string of length k has up to 2^k subsequences, one for each subset of its positions, so brute force needs about $2^{\min\{|S|,|T|\}}$ checks.

7.4 Edit distance

How do we compare two strings? Option 1: transform one string into the other. Option 2: look at their subsequences. The two options turn out to be closely related.

Definition 7.4: Edit distance

Given two sequences S and T, the *edit distance* $\mathrm{ED}(S, T)$ is the minimum number of insertions and deletions of characters needed to transform S into T.

Example 24: Transforming a string

Let $S =$ `CATGA` and $T =$ `ACTGC`:

$$\texttt{CATGA} \xrightarrow{\text{delete C}} \texttt{ATGA} \xrightarrow{\text{insert C}} \texttt{ACTGA} \xrightarrow{\text{insert C}} \texttt{ACTGAC} \xrightarrow{\text{delete A}} \texttt{ACTGC}.$$

Four changes suffice, and in fact $\mathrm{ED}(S, T) = 4$.

We could also delete all the letters one by one and insert the new ones, but that is not the shortest way. When the strings are longer, it is hard to see the minimum number of steps.

The edit distance is symmetric, $\mathrm{ED}(S,T) = \mathrm{ED}(T,S)$, because we can perform all the steps in reverse: each insertion becomes a deletion and vice versa.

Note: Substitutions

Some definitions also allow *substitutions*, replacing one character with another at cost 1. With insertions and deletions only, a substitution costs 2: one deletion and one insertion.

7.4.1 Edit distance and LCS

A natural strategy for transforming S into T: find an LCS, delete everything else from S, leaving only the LCS, and then insert the characters of T that are missing.

Proposition 7.1: Edit distance from the LCS

$$\mathrm{ED}(S,T) = \underbrace{\big(|S| - \mathrm{LCS}(S,T)\big)}_{\text{characters to delete}} + \underbrace{\big(|T| - \mathrm{LCS}(S,T)\big)}_{\text{characters to insert}} = |S| + |T| - 2\,\mathrm{LCS}(S,T).$$

Why it holds. The strategy above uses exactly $|S| + |T| - 2\,\mathrm{LCS}(S,T)$ operations, so $\mathrm{ED}(S,T)$ is at most this number. Conversely, in any sequence of insertions and deletions, the characters of S that are never deleted appear in T in the same order: they form a common subsequence, of some length $k \le \mathrm{LCS}(S,T)$. We must then delete at least $|S| - k$ characters and insert at least $|T| - k$, so at least $|S| + |T| - 2k \ge |S| + |T| - 2\,\mathrm{LCS}(S,T)$ operations are needed. □

For Example 24, $\mathrm{LCS}(\texttt{CATGA}, \texttt{ACTGC}) = 3$ (for example `CTG`), so $\mathrm{ED} = 5 + 5 - 2 \cdot 3 = 4$.

7.5 A recursive formula for the edit distance

The optimal algorithm for computing the edit distance is not obvious. We now write a recursive formula, the starting point of *dynamic programming*.

Note: Notation

Consider two strings S and T with $|S| = n$ and $|T| = m$:

$$S = [S[1], S[2], \ldots, S[n]], \qquad T = [T[1], \ldots, T[m]].$$

We write $S^i = [S[1], \ldots, S[i]]$ and $T^j = [T[1], \ldots, T[j]]$ for the *prefixes* of length i and j.

Proposition 7.2: Recurrence for the edit distance

$$\mathrm{ED}(S^i, T^j) = \begin{cases} j, & \text{if } i = 0, \\ i, & \text{if } j = 0, \\ \mathrm{ED}(S^{i-1}, T^{j-1}), & \text{if } S[i] = T[j], \\ \min\{\mathrm{ED}(S^i, T^{j-1}),\ \mathrm{ED}(S^{i-1}, T^j)\} + 1, & \text{if } S[i] \neq T[j]. \end{cases}$$

- If one string is empty, we must insert (or delete) all the characters of the other.
- If the last characters are equal, we do not need to modify them: we can ignore them and consider the distance between S^{i-1} and T^{j-1}. For example,

$$\mathrm{ED}(\texttt{CATGCAT T}, \texttt{ACCTAGAC T}) = \mathrm{ED}(\texttt{CATGCAT}, \texttt{ACCTAGAC}).$$

- If the last characters differ, one of them must go: either we remove the last character of S, giving $\mathrm{ED}(S^{i-1}, T^j) + 1$, or we remove the last character of T (equivalently, we insert it at the end of S), giving $\mathrm{ED}(S^i, T^{j-1}) + 1$. For `CATGCAT T` and `ACCTAGAC A`, we remove either the `T` or the `A`. We take the better of the two options.

Example 25: Small edit distances

- $\mathrm{ED}(\texttt{TGA}, \texttt{TTA}) = 2$: we must remove `G` and add `T`. Also $\mathrm{ED}(\texttt{TG}, \texttt{TT}) = 2$, because we have to remove a character and then add one.
- For `CAG` and `TGA`, the last characters differ. Either we remove `G` from `CAG`, which leaves `CA` and `TGA` (+1); or we remove `A` from `TGA`, which leaves `CAG` and `TG` (+1), whose last characters are now equal. Both options lead to $\mathrm{ED}(\texttt{CAG}, \texttt{TGA}) = 4$.

Algorithm 7.1: Recursive edit distance

```
1: function ED(S^i, T^j)
2:     if i = 0 then
3:         return j
4:     else if j = 0 then
5:         return i
6:     else if S[i] = T[j] then
7:         return ED(S^{i-1}, T^{j-1})
8:     else
9:         return 1 + min{ED(S^i, T^{j-1}), ED(S^{i-1}, T^j)}
10:    end if
11: end function
```

Written directly as a recursive algorithm, this formula has an exponential running time.

Note: The same problem as Fibonacci

As with the recursive Fibonacci algorithm (Proposition 3.1), the recursion solves the same subproblems again and again. There are only $(n+1)(m+1)$ different subproblems $\mathrm{ED}(S^i, T^j)$, so storing each value once (dynamic programming) avoids the repeated work.

8

Memory and data representation

Start of lecture 19 October

8.1 Bytes, addresses and consecutive memory locations

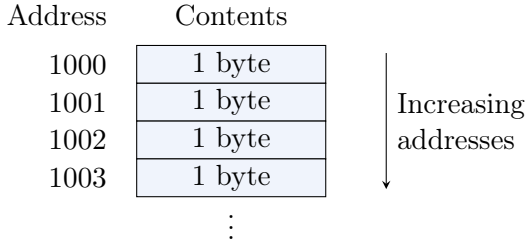
What is memory for? While a program runs, it needs somewhere to keep the data it is working with: numbers, text, lists and the intermediate results of calculations. Main memory (RAM) provides this working space, so the processor can read the data, perform operations and store the results for later steps of the program.

For example, after executing `x = 5`, Python must keep the object representing 5 available. A later instruction such as `print(x)` can retrieve it, and `x + 2` can use it to compute 7. Understanding how data are arranged and reached in memory will help us explain the cost of operations such as accessing an element of a list.

Definition 8.1: Bit and byte

A *bit* is a binary digit, either 0 or 1. A *byte* consists of eight bits.

Imagine memory as numbered cells, drawn vertically below. Each cell holds one byte. The number on the left is its address: it identifies the cell, while the contents are stored inside it.



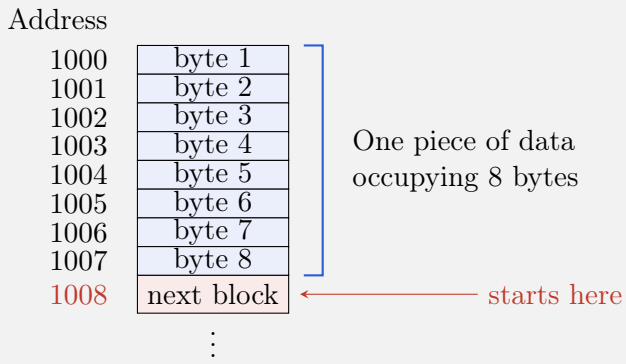
The address and the contents are different things: the label 1000 tells us where the first displayed byte is, not what value it holds. We use illustrative decimal addresses throughout these drawings.

Definition 8.2: Memory address

In byte-addressable memory, a *memory address* identifies the location of one byte. The address specifies *where* data are stored; the contents are the data stored at that location.

Example 26: A block occupying eight consecutive bytes

Suppose a piece of data occupies eight bytes starting at address 1000. Follow the addresses down the drawing:



The starting byte is already the first of the eight, so the last byte is at $1000 + 7 = 1007$. The first address *after* the block is

$$1000 + 8 = \boxed{1008}.$$

If a second eight-byte block starts immediately afterwards, its starting address is 1008.

This explains why a memory diagram can show addresses advancing in blocks: it may label the start of each block rather than drawing every individual byte. For eight-byte blocks placed consecutively, the starting addresses would be $1000, 1008, 1016, \ldots$.

8.2 Python names refer to objects

Now consider the Python assignment `x = 5`. We distinguish the name `x`, the object representing the integer 5, and the address where that object is stored.

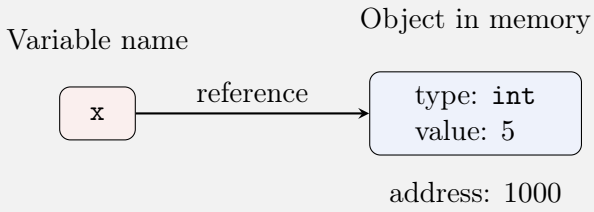
Definition 8.3: Reference to an object

A Python variable name is associated with an object through a *reference*. In our diagrams, an arrow from the name to the object represents that reference.

Example 27: Following the reference in `x = 5`

Step 1: an object represents the integer 5. The drawing shows its type, `int`, and its value, 5.

Step 2: the name `x` is associated with that object. The arrow lets us follow the reference from the name to the object.



The address 1000 is illustrative. It identifies the object's location, while 5 is the object's value and `x` is its name in this example.

Step 3: evaluate `print(x)`. Python looks up the object associated with `x`. Following the arrow reaches the integer object with value 5, so the output is `5`.

9 Text mining

Start of lecture 20 October

9.1 From raw text to terms

A search engine receives a query and looks for the top k most related documents in a collection, or *corpus*. We will turn this into a nearest-neighbour problem, but first we need to turn each document's text into terms that we can compare and count.

Definition 9.1: Text preprocessing

Text preprocessing transforms a document, initially a sequence of characters, into a sequence of words or terms suitable for the analysis that follows.



1. Tokenization. Split the text into units called *tokens*. For example, the document

"Today is Monday and the weather is awesome."

can be split into the word tokens

`Today`, `is`, `Monday`, `and`, `the`, `weather`, `is`, `awesome`.

The input was a string of characters; the output is now a sequence of separate terms.

2. Stopword removal. Remove words on a chosen *stopword list*, such as `the`, `is`, `to` and `and`. These words are often too common to help distinguish document topics. Depending

on the list, a phrase such as "to be or not to be" can lose most of its words. We will later examine how common words affect cosine similarity.

3. Text normalization. Map different written forms to a common form when they should match. For example,

$$\texttt{U.S.A.} \longrightarrow \texttt{USA}, \qquad \texttt{Monday} \longrightarrow \texttt{monday}.$$

The first transformation removes punctuation within an abbreviation; the second ignores capitalization. Other choices concern accents and hyphens. This normalization acts on the text; the vector normalization introduced later rescales numerical coordinates.

4. Stemming. Reduce related word forms to a common *stem*, using rules that remove affixes such as suffixes. For example,

$$\texttt{playing}, \texttt{plays}, \texttt{play} \quad \longrightarrow \quad \texttt{play}.$$

The aim is to count these forms together instead of treating each as a completely separate term. `NLTK` is a Python toolkit for text-processing operations.

9.2 From documents to vectors

How can we turn a text into something on which we can perform calculations? Suppose we want to find documents about similar topics. Before choosing a distance or a similarity function, we need a numerical representation of each document.

Definition 9.2: Corpus and vocabulary

A *corpus* is a collection of documents. Its *vocabulary* is the set of distinct terms appearing in those documents, after any preprocessing. We fix an order for the vocabulary and use the same order for every document.

Example 28: Counting words in two documents

Consider the documents

$$D_1: \quad \text{"today we started at three"},$$
$$D_2: \quad \text{"we played three by three"}.$$

For this example, we keep all the words. In the vocabulary order

$$(\texttt{at}, \texttt{by}, \texttt{played}, \texttt{started}, \texttt{three}, \texttt{today}, \texttt{we}),$$

their word counts are

	at	by	played	started	three	today	we
D_1	1	0	0	1	1	1	1
D_2	0	1	1	0	2	0	1

Each entry counts how many times a term occurs in a document. For example, the entry for `three` in D_2 is 2, because that word appears twice. Reading the rows gives the vectors

$$\mathbf{d}_1 = (1, 0, 0, 1, 1, 1, 1), \qquad \mathbf{d}_2 = (0, 1, 1, 0, 2, 0, 1).$$

The common coordinate order is essential: a coordinate must refer to the same word in both documents for a comparison to make sense.

Definition 9.3: Count-vector representation

Let the ordered vocabulary be $(t_1, \ldots, t_m)$. We represent document D_i by the vector $\mathbf{d}_i \in \mathbb{R}^m$, where

$$d_{ij} = \text{number of occurrences of term } t_j \text{ in document } D_i.$$

Thus every term defines one coordinate, and every document becomes a point in the same m-dimensional space. This is a basic *vector space model* of documents.

This representation records which terms occur and how often, but it does not preserve their order in the text. Consequently, identical count vectors do not necessarily imply identical meanings.

9.3 Why document length can distort a comparison

Now that documents are points, we can compare them using the L_2 distance already introduced in Definition 6.2. For a two-dimensional illustration, keep only the coordinates (`three`, `we`) from Example 28:

$$\mathbf{u}_1 = (1, 1), \qquad \mathbf{u}_2 = (2, 1).$$

These are two-coordinate views of the documents, rather than their complete vectors. Their distance is

$$\|\mathbf{u}_1 - \mathbf{u}_2\|_2 = \sqrt{(1-2)^2 + (1-1)^2} = 1.$$

We would like closeness between points to reflect similarity between documents. However, raw word counts introduce a difficulty.

Example 29: Repeating a document ten times

Here D_1 is the original five-word document:

$$D_1 : \quad \text{“today we started at three”}.$$

Construct D_3 by repeating this *entire sentence ten times inside a single document*:

$$D_3 : \quad \underbrace{\text{“today we started at three”} \cdots \text{“today we started at three”}}_{\text{10 copies of the sentence, 50 words in total}}.$$

Using the same vocabulary order as before, compare the counts:

	at	by	played	started	three	today	we
D_1	1	0	0	1	1	1	1
D_3	10	0	0	10	10	10	10

Thus D_3 is one longer document, not a collection of ten separate documents. Every term now occurs ten times as often:

$$\mathbf{d}_3 = 10\mathbf{d}_1.$$

In particular, `three` occurs ten times and `we` occurs ten times, so the two-coordinate view becomes $\mathbf{u}_3 = (10, 10)$. Its distance from the original is

$$\|\mathbf{u}_1 - \mathbf{u}_3\|_2 = \sqrt{(1-10)^2 + (1-10)^2} = 9\sqrt{2}.$$

Yet D_3 repeats exactly the same content: it adds no new information and does not change the topic. Every word retains the same proportion of the total text. The large distance is caused by the increase in length, rather than by a change in word composition.

9.4 Removing the effect of length: normalization

The idea is to rescale the counts so that multiplying every count by the same factor does not change the representation. A first way to do this is to divide each count by the total number of terms in the document: we then compare *relative frequencies*.

Example 30: From counts to proportions

As in Example 29, the documents are

$$D_1: \quad \text{“today we started at three”},$$

$$D_3: \quad \underbrace{\text{“today we started at three”} \cdots \text{“today we started at three”}}_{\text{10 copies of the sentence, 50 words in total}}.$$

	at	by	played	started	three	today	we
D_1	1	0	0	1	1	1	1
D_3	10	0	0	10	10	10	10

Document D_1 contains five words and D_3 contains fifty. For the coordinates (`three`, `we`), dividing by the respective document lengths gives

$$D_1: \quad \left(\frac{1}{5}, \frac{1}{5}\right) = (0.2, 0.2), \qquad D_3: \quad \left(\frac{10}{50}, \frac{10}{50}\right) = (0.2, 0.2).$$

The representations now agree. Notice that the denominators count *all* the terms in each document, not just the two displayed coordinates.

More generally, we can divide a nonzero document vector by its norm.

Definition 9.4: Document-vector normalization

For a nonzero vector $\mathbf{d}$ and a chosen norm, its normalized version is

$$\widehat{\mathbf{d}} = \frac{\mathbf{d}}{\|\mathbf{d}\|}.$$

Each coordinate is divided by the same positive number, and the resulting vector has norm 1.

For a complete vector of nonnegative term counts, the L_1 norm is the sum of all counts, hence the total number of terms counted in the document. Normalizing with L_1 therefore gives the relative frequencies used above. Normalizing with L_2 instead gives a vector of Euclidean norm 1; these are two different choices of scale.

Both choices remove a uniform multiplication of the counts. Indeed, for $c > 0$ and $\mathbf{d} \neq \mathbf{0}$,

$$\frac{c\mathbf{d}}{\|c\mathbf{d}\|} = \frac{c\mathbf{d}}{c\|\mathbf{d}\|} = \frac{\mathbf{d}}{\|\mathbf{d}\|}.$$

Thus repeating D_1 ten times yields the same normalized vector. Geometrically, normalization rescales the vector without changing its direction. A document with no counted terms has the zero vector and cannot be normalized by this formula.

9.5 Comparing directions: cosine similarity

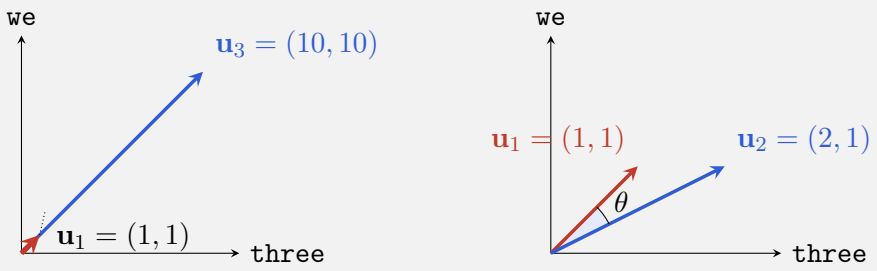
Normalization suggests a geometric way to compare documents: two vectors can have different lengths while pointing in the same direction. Now, we can consider another idea: compare the *angle* between the vectors. If their counts are proportional, their relative word composition is the same and the angle is zero.

Example 31: Same direction, different lengths

As before, keep only the coordinates (`three`, `we`):

$$\mathbf{u}_1 = (1, 1), \qquad \mathbf{u}_3 = (10, 10).$$

Draw each vector as an arrow starting at the origin. The arrow for $\mathbf{u}_3$ is ten times as long, but points in exactly the same direction as $\mathbf{u}_1$: the angle between them is $0°$. Now consider $\mathbf{u}_2 = (2, 1)$. Here `three` occurs twice as often as `we`, so its arrow points more towards the `three` axis and forms a positive angle with $\mathbf{u}_1$.



The smaller the angle, the more similar the proportions of terms in the two vectors. We turn this idea into a numerical similarity by taking the cosine of that angle.

Definition 9.5: Cosine similarity

For two nonzero document vectors $\mathbf{u}$ and $\mathbf{v}$, let θ be the angle between them. Their *cosine similarity* is

$$\mathrm{sim}_{\cos}(\mathbf{u}, \mathbf{v}) = \cos\theta.$$

For nonnegative count vectors, the angle lies between 0° and 90°, so the similarity lies between 0 and 1:

- At 0°, the similarity is 1: the vectors point in the same direction, and their counts are proportional.
- Between 0° and 90°, the similarity lies strictly between 1 and 0. As the angle increases, the similarity decreases.
- At 90°, the similarity is 0: there is no coordinate with a positive count in both vectors. For complete document vectors, this means that the documents share no counted terms.

In particular, D_1 and its tenfold repetition D_3 have cosine similarity 1. This measures similarity of word composition; as with the count representation itself, it does not capture word order or guarantee identical meanings. A zero vector has no direction, so the similarity above is not defined for an empty count vector.

9.5.1 Computing cosine similarity from coordinates

We do not need to measure the angle first. For two nonzero vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^m$, we can compute its cosine directly from their coordinates:

$$\mathrm{sim}_{\cos}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} = \frac{\sum_{j=1}^{m} u_j v_j}{\sqrt{\sum_{j=1}^{m} u_j^2}\sqrt{\sum_{j=1}^{m} v_j^2}}.$$

The numerator is the *dot product*: multiply corresponding coordinates, then add the results. The denominator is the product of the two L_2 norms.

Example 32: Computing the similarity step by step

Take the two vectors from the right-hand graph, in the coordinate order (`three`, `we`):

$$\mathbf{u}_1 = (1, 1), \qquad \mathbf{u}_2 = (2, 1).$$

Step 1: compute the dot product. Multiply the counts for the same word and add:

$$\mathbf{u}_1 \cdot \mathbf{u}_2 = \underbrace{1 \cdot 2}_{\texttt{three}} + \underbrace{1 \cdot 1}_{\texttt{we}} = 3.$$

Each term compares the count of one word in the first document with the count of that same word in the second.

Step 2: compute the two L_2 norms.

$$\|\mathbf{u}_1\|_2 = \sqrt{1^2 + 1^2} = \sqrt{2}, \qquad \|\mathbf{u}_2\|_2 = \sqrt{2^2 + 1^2} = \sqrt{5}.$$

Step 3: divide the dot product by the product of the norms.

$$\mathrm{sim}_{\cos}(\mathbf{u}_1, \mathbf{u}_2) = \frac{3}{\sqrt{2}\sqrt{5}} = \frac{3}{\sqrt{10}} \approx 0.949.$$

The result is close to 1: the vectors point in similar directions, although their word proportions differ. This is the cosine of the angle of approximately 18.4° shown in the graph.

9.5.2 A limitation: matching words is not understanding meaning

Two texts can discuss the same subject while using different terms, such as `cat` and `feline`. In our representation these are separate coordinates: a shared meaning does not create a shared term. Conversely, the same word can have different meanings in different contexts. The count-based model does not resolve these ambiguities by itself. The next weighting step changes the importance of terms, but does not make distinct synonyms the same coordinate.

9.6 Why some words should count more than others

We can now compare documents using cosine similarity. However, in the count representation every occurrence has the same weight, regardless of which word it belongs to. A common word can therefore create a similarity that tells us little about the documents' topics.

Example 33: A similarity caused by a common word

Consider two short documents:

$$D_A: \quad \text{“the cat”}, \qquad D_B: \quad \text{“the dog”}.$$

Using the vocabulary order (`the`, `cat`, `dog`), we obtain

$$\mathbf{d}_A = (1, 1, 0), \qquad \mathbf{d}_B = (1, 0, 1).$$

Their dot product is

$$\mathbf{d}_A \cdot \mathbf{d}_B = \underbrace{1 \cdot 1}_{\texttt{the}} + \underbrace{1 \cdot 0}_{\texttt{cat}} + \underbrace{0 \cdot 1}_{\texttt{dog}} = 1.$$

Both vectors have L_2 norm $\sqrt{2}$, so

$$\mathrm{sim}_{\cos}(\mathbf{d}_A, \mathbf{d}_B) = \frac{1}{\sqrt{2}\sqrt{2}} = \frac{1}{2}.$$

The similarity comes entirely from `the`, a word that provides little information about what these documents discuss.

A first solution is to remove *stopwords*: very common words, such as articles and prepositions, that are often uninformative for the comparison. In the example, removing `the` leaves

$$D_A: \quad \text{“cat”}, \qquad D_B: \quad \text{“dog”}.$$

With the vocabulary order (`cat`, `dog`), the vectors become

$$\mathbf{d}_A = (1, 0), \qquad \mathbf{d}_B = (0, 1).$$

Their dot product is zero and both norms are 1, so their cosine similarity is now 0: they share no remaining terms.

This suggests a more general idea: words are not all equally informative. Instead of only removing certain words, we can assign different weights to the terms, so that informative words contribute more to the comparison.

9.7 Weighting terms by their rarity: IDF

A word that appears in almost every document does little to distinguish documents from one another. A word found in only a few documents can be more characteristic. For example, in a corpus of cooking texts, `food` might occur almost everywhere, whereas `guanciale` might appear only in recipes using that ingredient.

Example 34: Counting documents rather than occurrences

Suppose our corpus contains 100 documents:

Term	Documents containing the term
`food`	100
`pasta`	20
`guanciale`	2

These are counts of *documents*, not occurrences. If `guanciale` occurs ten times in one recipe, that recipe still contributes only 1 to the count of documents containing the term.

Step 1: count the documents containing each term. Let N be the total number of documents. For a term t, write $\mathrm{df}(t)$ for its *document frequency*: the number of documents containing it.

Step 2: compare the total with that count. Consider the ratio $N/\mathrm{df}(t)$. In our example,

$$\texttt{food}: \ \frac{100}{100} = 1, \qquad \texttt{pasta}: \ \frac{100}{20} = 5, \qquad \texttt{guanciale}: \ \frac{100}{2} = 50.$$

The rarer the term in the corpus, the larger this ratio.

Step 3: take the logarithm of the ratio. The logarithm reduces the differences between the weights while preserving their order: rarer terms still receive larger weights.

Definition 9.6: Inverse document frequency

For a term t appearing in at least one document of a corpus of N documents, its *inverse document frequency* is

$$\mathrm{IDF}(t) = \log\left(\frac{N}{\mathrm{df}(t)}\right),$$

where $\mathrm{df}(t)$ is the number of documents containing t.

Example 35: IDF weights for the cooking corpus

Using base-10 logarithms, the weights are

Term	$N/\mathrm{df}(t)$	$\mathrm{IDF}(t)$
`food`	$100/100 = 1$	$\log_{10}(1) = 0$
`pasta`	$100/20 = 5$	$\log_{10}(5) \approx 0.699$
`guanciale`	$100/2 = 50$	$\log_{10}(50) \approx 1.699$

Thus `guanciale` receives more weight than `pasta`. The word `food`, present in every document, receives weight 0: under this formula it provides no information for distinguishing the documents in this corpus.

9.8 Combining term frequency and rarity: TF-IDF

We now combine two pieces of information. The *term frequency* (TF) describes how often a term occurs in one particular document; IDF describes how rare that term is across the entire corpus. Here we use the raw occurrence count as TF, consistently with our count vectors.

Definition 9.7: TF-IDF weight

Let $\mathrm{TF}(t, D)$ be the number of occurrences of term t in document D. Its *TF-IDF weight* is

$$\text{TF-IDF}(t, D) = \mathrm{TF}(t, D) \cdot \mathrm{IDF}(t).$$

Example 36: From counts to TF-IDF weights

Step 1: keep the same corpus of 100 documents. Using the base-10 logarithms from the previous example,

$$\mathrm{IDF}(\texttt{food}) = 0, \qquad \mathrm{IDF}(\texttt{pasta}) \approx 0.699, \qquad \mathrm{IDF}(\texttt{guanciale}) \approx 1.699.$$

Step 2: choose a document D in this corpus. Suppose its counts for these terms are

Term	TF in document D
`food`	4
`pasta`	3
`guanciale`	2

In the coordinate order $(\texttt{food}, \texttt{pasta}, \texttt{guanciale})$, the corresponding count vector is $\mathbf{d} = (4, 3, 2)$. These are the three coordinates we are following in the example.

Step 3: multiply each count by its term's IDF.

Term	TF	IDF	TF-IDF
`food`	4	0	$4 \cdot 0 = 0$
`pasta`	3	≈ 0.699	$3 \log_{10}(5) \approx 2.097$
`guanciale`	2	≈ 1.699	$2 \log_{10}(50) \approx 3.398$

The weighted vector is therefore

$$\mathbf{d}_{\text{TF-IDF}} \approx (0,\ 2.097,\ 3.398).$$

Although `guanciale` occurs less often than `pasta` in this document, it receives a larger weight because it is rarer in the corpus. The term `food` receives weight zero because it appears in every document.

TF depends on the particular document being represented. IDF depends on the corpus: for a fixed corpus, a term has the same IDF in every document. We can then compare the weighted document vectors using the same cosine-similarity formula as before, provided neither weighted vector is zero.

9.9 Using document vectors for search

We can use TF-IDF vectors to build a search engine. A *query* is the text entered by the user. Given a query and a positive integer k, our goal is to return the k documents most similar to that query (or all documents if the corpus contains fewer than k). For example, $k = 3$ asks for the three highest-scoring results.

The first step is to represent the query as a vector in the *same space* as the documents: the same vocabulary, the same coordinate order and the same IDF weights.

Example 37: Turning a query into a vector

Suppose the user searches for

$$Q: \quad \text{“pasta guanciale”}.$$

Keep the coordinate order (`food`, `pasta`, `guanciale`) from the cooking example.

Step 1: count the query terms. The query contains no occurrence of `food`, one occurrence of `pasta` and one of `guanciale`. Its count vector is

$$\mathbf{q}_{\text{counts}} = (0, 1, 1).$$

Step 2: apply the same IDF weights as for the documents. Use the IDF values already computed from the corpus, rather than recomputing IDF from the query:

$$\mathbf{q}_{\text{TF-IDF}} \approx (0 \cdot 0,\ 1 \cdot 0.699,\ 1 \cdot 1.699) = (0,\ 0.699,\ 1.699).$$

As before, we display the three coordinates being followed in this example; any other vocabulary terms absent from the query have coordinate zero.

Step 3: compare the query with each document. For each document D_i, assign the score

$$\text{score}(D_i) = \text{sim}_{\cos}\big(\mathbf{q}_{\text{TF-IDF}}, \mathbf{d}_{i,\text{TF-IDF}}\big).$$

Here both vectors use the full common vocabulary and are assumed to be nonzero. A higher score means that the document's direction is closer to the query's direction. Sort the documents by decreasing score and return the first k.

The query is therefore treated as a small document. Representing it with the same coordinates and corpus-based IDF weights lets us use the same cosine-similarity calculation for retrieval.

9.9.1 The cost of searching every document

The simplest search procedure follows four steps:

1. Compute cosine similarity between the query and every document.
2. Store each document's score.

3. Sort the documents by decreasing score.
4. Return the first k results.

Suppose the corpus contains N documents and the vocabulary contains m terms. We analyse the direct implementation that scans all m coordinates of each document vector.

First cost: computing all scores. Each vector has m coordinates. Scanning all of them to compare the query with one document takes $\Theta(m)$ time. Repeating this for all N documents takes

$$\Theta(Nm).$$

Example 38: A large corpus

With one million documents and a vocabulary of one hundred thousand terms, the direct implementation examines

$$10^6 \cdot 10^5 = 10^{11}$$

pairs of coordinates for every query. Each pair contains the query weight and the document weight for the same vocabulary term.

Second cost: sorting the scores. One simple approach repeatedly finds the largest remaining score, places its document next in the ordering and removes it from consideration. Producing the complete ordering this way scans

$$N + (N-1) + \cdots + 1 = \frac{N(N+1)}{2}$$

scores, so it takes $\Theta(N^2)$ time.

An efficient comparison-based sorting algorithm sorts the N scores in $O(N \log N)$ time. Thus computing all scores and sorting them gives the overall bound

$$O(Nm + N \log N).$$

This procedure works, but repeats substantial work whenever a new query arrives. The next question is which work can be performed once, while preparing the documents, and which must wait until the query is known.

9.9.2 Preparing the corpus offline, answering queries online

The corpus is usually available before the user submits a query. We can therefore prepare information about the documents in advance, instead of repeating all the preparation for every search.

Definition 9.8: Offline and online phases

The *offline phase* processes the corpus before a particular query is known, storing information useful for future searches. The *online phase* starts when a query arrives and uses that stored information to compute the results.

Here "offline" means independent of the current query, rather than disconnected from the Internet. For our vector-based search, we can prepare the vocabulary, document counts, IDF weights and document TF-IDF vectors offline. We can also store their L_2 norms, or their normalized vectors. Online, we process the query, construct its vector using the stored vocabulary and IDF, and compute the results.

Example 39: Many users searching the same corpus

Suppose one user searches for "pasta guanciale" and another for "tomato restaurant". Their query vectors differ, but the document vectors and corpus-based IDF weights are the same. We prepare those once and reuse them for both searches. If the corpus changes, the stored information must be updated as appropriate.

Precomputing document vectors saves preparation work, but scanning every document still has a cost. To understand how a different data organization can help, we now consider a Boolean version of search.

9.10 Boolean AND search

Suppose the query is "pasta guanciale". This time we want every document containing *both* words. A document containing only `pasta` does not qualify, however often that word occurs.

Definition 9.9: Boolean AND query

Represent each document D_i by its set of terms T_i, and let Q be a nonempty set of query terms. A Boolean AND query returns

$$R(Q) = \{i : Q \subseteq T_i\}.$$

Thus a document qualifies precisely when it contains every query term.

This search asks a membership question, rather than assigning cosine scores and selecting the top k. Its output is the set of all qualifying document identifiers.

9.10.1 A direct scan

A first approach to answering this query is to visit each document, check whether it contains every query term, and add its identifier to the result if it does. For a large corpus this inspects many documents that contain none of the query terms. We would like to find the relevant documents without starting a new scan of the entire corpus each time.

9.11 The inverted index

The usual document representation answers: "Which terms occur in this document?" We now reverse the direction and store the answer to: "Which documents contain this term?"

Definition 9.10: Inverted index and posting list

An *inverted index* maps each term t to its *posting list* $L(t)$: the sorted list of identifiers of documents containing t. Each document identifier appears at most once in a term's list, regardless of how often that term occurs in the document.

Example 40: Building the index from six documents

Use letters to represent terms, and number the documents from 1 to 6. The original representation is

Document identifier	Terms in the document
1	A, B, C, D
2	A, B, E, G
3	B, D, F
4	A, C, D, E
5	A, C, G
6	C, D, G

Step 1: collect the documents containing A. Reading the table, A occurs in documents $1, 2, 4, 5$, so $L(A) = [1, 2, 4, 5]$.

Step 2: repeat for every term. The complete inverted index for these six documents is

Term	Posting list
A	$[1, 2, 4, 5]$
B	$[1, 2, 3]$
C	$[1, 4, 5, 6]$
D	$[1, 3, 4, 6]$
E	$[2, 4]$
F	$[3]$
G	$[2, 5, 6]$

For a single-term query E, the answer is immediately $\{2, 4\}$. We look up $L(E)$ rather than checking all six documents.

We build the inverted index offline. One way is to process documents in increasing identifier order and append each identifier to the lists of its distinct terms; the lists are then already sorted. Online, a query only needs the lists for its own terms.

9.11.1 AND means intersecting posting lists

For the query $Q = \{A, C, E\}$, a document must occur in all three lists:

$$R(Q) = L(A) \cap L(C) \cap L(E).$$

Here we use intersection to mean the identifiers shared by the lists. Using Example 40,

$$\{1, 2, 4, 5\} \cap \{1, 4, 5, 6\} = \{1, 4, 5\}, \qquad \{1, 4, 5\} \cap \{2, 4\} = \{4\}.$$

The answer is document 4. Looking back at its row confirms that it contains A, C, E. If any query term has no posting list, no document contains that term and the AND result is empty.

9.12 Intersecting sorted lists with pointers

How can we find the intersection without repeatedly searching one list for every element of another? Sorting lets us move through the lists from left to right, using a *pointer* to mark the current position in each list.

Example 41: Following the pointers

The query is $Q = \{A, C, E\}$: we want documents containing all three terms. The posting lists are

$$L(A) = [1, 2, 4, 5], \quad L(C) = [1, 4, 5, 6], \quad L(E) = [2, 4].$$

Each number identifies a document. Put a pointer at the first element of each list; the boxed entries below show the current positions.

Step 1: the pointers read $1, 1, 2$.

$$L(A) = [\boxed{1}, 2, 4, 5],$$
$$L(C) = [\boxed{1}, 4, 5, 6],$$
$$L(E) = [\boxed{2}, 4].$$

Document 1 contains A and C, but not E. In $L(E)$ we are already at 2; since the list is sorted, no later entry can be 1. Thus 1 cannot be a result. Advance the two pointers reading 1 (those in $L(A)$ and $L(C)$), leaving the pointer in $L(E)$ at 2.

Step 2: the pointers now read $2, 4, 2$.

$$L(A) = [1, \boxed{2}, 4, 5],$$
$$L(C) = [1, \boxed{4}, 5, 6],$$
$$L(E) = [\boxed{2}, 4].$$

Document 2 contains A and E, but not C. In $L(C)$ we are already at 4, so we cannot find 2 further along that list. Discard 2: advance the pointers in $L(A)$ and $L(E)$, leaving the pointer in $L(C)$ at 4.

Step 3: the pointers now read $4, 4, 4$.

$$L(A) = [1, 2, \boxed{4}, 5],$$
$$L(C) = [1, \boxed{4}, 5, 6],$$
$$L(E) = [2, \boxed{4}].$$

All three pointers read the same identifier: document 4 contains A, C and E. Add it to the result, giving $R = [4]$, then advance all three pointers.

Step 4: the list for E is exhausted.

$$L(A) = [1, 2, 4, \boxed{5}],$$
$$L(C) = [1, 4, \boxed{5}, 6],$$
$$L(E) = [2, 4] \quad \text{(pointer past the last entry)}.$$

The other pointers read 5, but $L(E)$ has no entries left. No further identifier can occur in all three lists, so we stop. The final result is $R = [4]$: return document D_4.

Summary of the pointer movements.

Step	$L(A)$	$L(C)$	$L(E)$	Action
1	1	1	2	Advance the pointers at 1.
2	2	4	2	Advance the pointers at 2.
3	4	4	4	Output 4; advance all three.
4	5	5	exhausted	Stop.

For a fixed small number of lists, this gives a simple rule: if all current identifiers agree, output that identifier and advance every pointer; otherwise advance the pointers at the smallest current identifier. No pointer needs to move backwards.

9.12.1 A two-list intersection algorithm

For two lists, there are only three cases: the current identifiers are equal, the left one is smaller, or the right one is smaller. The following algorithm makes each case explicit.

Algorithm 9.1: Intersect two sorted posting lists

```
1: function INTERSECT(L, M)
2:     i ← 1, j ← 1, R ← [ ]
3:     while i ≤ |L| and j ≤ |M| do
4:         if L[i] = M[j] then
5:             Append L[i] to R
6:             i ← i + 1, j ← j + 1
7:         else if L[i] < M[j] then
8:             i ← i + 1
9:         else
10:            j ← j + 1
11:        end if
12:    end while
13:    return R
14: end function
```

Every iteration advances at least one pointer, and each pointer crosses its list at most once. Thus intersecting lists of lengths a and b takes $O(a+b)$ time, with $\Theta(a+b)$ in the worst case. It can finish sooner if one list is exhausted early.

9.12.2 Query cost: the lengths of the relevant lists

We now measure the work needed to answer a query after the inverted index has already been built. Let $n_t = |L(t)|$ be the length of the posting list for query term t: the number of documents containing that term. We work with the lists selected by the query.

For several terms, we can apply the two-list algorithm repeatedly. Start with the shortest posting list and intersect the current result with each of the other lists. Stop if the result becomes empty.

Example 42: The cost of a three-term query

Take $Q = \{A, C, E\}$, with

$$L(A) = [1, 2, 4, 5], \quad L(C) = [1, 4, 5, 6], \quad L(E) = [2, 4].$$

Step 1: count the lengths of the relevant lists.

Term	Posting-list length
A	$n_A = 4$
C	$n_C = 4$
E	$n_E = 2$

Their sum is $n_A + n_C + n_E = 4 + 4 + 2 = 10$. The lists for B, D, F, G are not needed for this query.

Step 2: start with the shortest list, $L(E)$, and intersect it with $L(A)$.

$$[2, 4] \cap [1, 2, 4, 5] = [2, 4].$$

The pairs of current identifiers compared by the two pointers are

$$(2,1) \longrightarrow (2,2) \longrightarrow (4,4).$$

First advance past 1 in $L(A)$. Then output 2 and advance both pointers; next output 4 and advance both again. The first list is now exhausted. We made three comparisons and obtained $R = [2,4]$.

Step 3: intersect the intermediate result with $L(C)$.

$$[2,4] \cap [1,4,5,6] = [4].$$

This time the comparisons are

$$(2,1) \longrightarrow (2,4) \longrightarrow (4,4).$$

Advance past 1 in $L(C)$, then past 2 in the intermediate result. Finally output 4 and advance both pointers. The intermediate list is exhausted. These are three more comparisons, and the final answer is document D_4.

Step 4: relate the work to the list lengths. The first intersection has the bound $O(2+4)$; the second also has the bound $O(2+4)$. The intermediate list never grows: it starts with at most two identifiers and intersections can only remove them. We performed six identifier comparisons in this example.

The general bound below is proportional to the sum of the original posting-list lengths, which here is 10.

Why starting with the shortest list gives the total bound. The intermediate result is always a subset of the shortest initial list, so its length never exceeds that list's length.

Let s be the length of the shortest initial list and n_t the length of the next posting list. The intermediate result has length at most s, so intersecting it with that next list costs $O(s+n_t)$. Since the next list cannot be shorter than the shortest initial list, $s \leq n_t$. Therefore

$$s + n_t \leq 2n_t, \qquad O(s+n_t) = O(n_t).$$

To see the total bound, let $r = |Q|$, the number of query terms and hence the number of posting lists. One list is used as the starting point, leaving $r - 1$ lists to intersect with the current result. Since each of these other lists has length at least s,

$$(r-1)s \le \sum_{t \in Q \setminus \{t_*\}} n_t,$$

where t_* is the term with the shortest list. Consequently the total intersection time is

$$O\left(\sum_{t \in Q} n_t\right).$$

To retrieve the lists, look up each of the r query terms in the inverted index. With a hash-table dictionary, each lookup takes $O(1)$ expected time, giving $O(r)$ expected time for all lookups. Retrieving a list here means obtaining a reference to the stored list, rather than copying or scanning its entries. Scanning the entries is counted in the intersection cost. The overall expected query cost is therefore

$$\boxed{O\left(\underbrace{r}_{\text{retrieve the lists}} + \underbrace{\sum_{t \in Q} n_t}_{\text{intersect the lists}}\right).}$$

In a much larger corpus, rare query terms can have short lists, making the online search far cheaper than a full corpus scan. If the terms occur in almost every document, their lists are long and the advantage is smaller.

10 Sorting

Start of lecture 26 October

10.1 The sorting problem

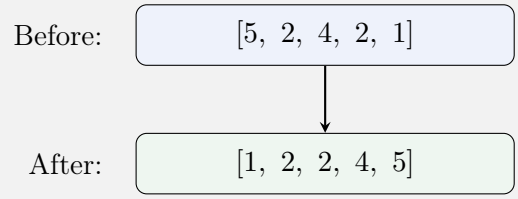
Sorting means arranging elements in order. We begin with a simple goal: given a list of numbers, arrange them from smallest to largest.

Problem: Sorting

Input: an array A of numbers, not necessarily sorted.

Output: an array containing the same elements as A in nondecreasing order, preserving every occurrence of each value.

Example 43: The same numbers, in a different order



The result must satisfy two conditions:

1. Each number is less than or equal to the next one.
2. The elements remain the same: only their order changes. In particular, both occurrences of 2 must remain in the result.

With five numbers, we can easily do this by hand. A computer needs precise instructions that also work for millions of elements. We will study different algorithms for the same sorting problem, examining how they work and how much time they require.

10.2 MergeSort

The first sorting algorithm we study is *MergeSort*. For example, we may start from

$$A = [2, 10, 7, 16, 3, 8, 9, 1, 11, 13, 14, 20, 17].$$

10.2.1 How Merge works: always take the smaller current element

Before studying the complete algorithm, we focus our attention on the function MERGE, a building block used by MergeSort.

Problem: Merging two sorted arrays

Input: two arrays A_1 and A_2, each already sorted in nondecreasing order.

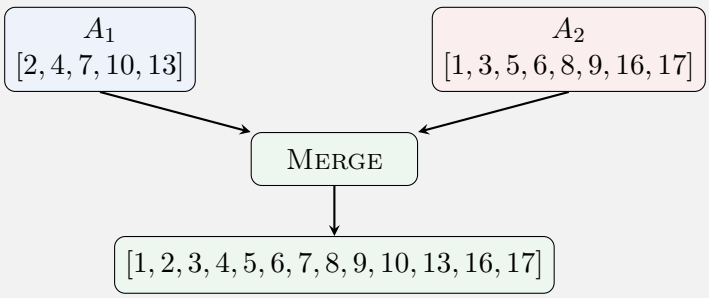
Output: one sorted array containing all the elements of both arrays, preserving every occurrence of each value.

Example 44: The input and output of Merge

Consider the following separate example of a call to MERGE(A_1, A_2):

$$A_1 = [2, 4, 7, 10, 13], \qquad A_2 = [1, 3, 5, 6, 8, 9, 16, 17].$$

We want one array containing all their values, in order:



The first array has 5 elements and the second has 8, so the output must contain $5 + 8 = 13$ elements.

Because both input arrays are sorted, the smallest element still available in each array is its first unprocessed element. To choose the next output value, we only need to compare those two elements.

Place a pointer i at the start of A_1 and a pointer j at the start of A_2. Create an empty output array B. At each step:

1. Compare $A_1[i]$ and $A_2[j]$.
2. Append the smaller value to B.
3. Advance only the pointer of the array from which we took it.

If the two values are equal, take the one from A_1 first. The other occurrence remains available in A_2, so neither occurrence is lost.

Example 45: Following Merge on the two arrays

Start with

$$A_1 = [2, 4, 7, 10, 13], \qquad A_2 = [1, 3, 5, 6, 8, 9, 16, 17], \qquad B = [\,].$$

Step 1: compare 2 and 1.

$$A_1 = [\boxed{2}, 4, 7, 10, 13], \qquad A_2 = [\boxed{1}, 3, 5, 6, 8, 9, 16, 17].$$

Take 1 from A_2: $B = [1]$. Advance its pointer to 3; the pointer in A_1 stays at 2.

Step 2: compare 2 and 3.

$$A_1 = [\boxed{2}, 4, 7, 10, 13], \qquad A_2 = [1, \boxed{3}, 5, 6, 8, 9, 16, 17].$$

Take 2 from A_1: $B = [1, 2]$. Advance its pointer to 4.

Step 3: compare 4 and 3.

$$A_1 = [2, \boxed{4}, 7, 10, 13], \qquad A_2 = [1, \boxed{3}, 5, 6, 8, 9, 16, 17].$$

Take 3 from A_2: $B = [1, 2, 3]$. Advance its pointer to 5.

Continue with the same rule. The following rows show the current values before each remaining comparison and the output after taking the smaller one:

$A_1[i]$	$A_2[j]$	Take	Output B
4	5	4	$[1, 2, 3, 4]$
7	5	5	$[1, 2, 3, 4, 5]$
7	6	6	$[1, 2, 3, 4, 5, 6]$
7	8	7	$[1, 2, 3, 4, 5, 6, 7]$
10	8	8	$[1, 2, 3, 4, 5, 6, 7, 8]$
10	9	9	$[1, 2, 3, 4, 5, 6, 7, 8, 9]$
10	16	10	$[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$
13	16	13	$[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 13]$

When one input is exhausted, copy the rest of the other. After taking 13, A_1 is exhausted. The remaining values in A_2 are 16, 17. They are already sorted and can be appended directly:

$$B = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 13, 16, 17].$$

The algorithm below uses indices starting at 1. The output array B has space for $|A_1|+|A_2|$ elements, and k indicates the next output position to fill.

Algorithm 10.1: Merge two sorted arrays

```
1: function MERGE(A_1, A_2)
2:     n_1 ← |A_1|, n_2 ← |A_2|
3:     Allocate an array B of length n_1 + n_2
4:     i ← 1, j ← 1, k ← 1
5:     while i ≤ n_1 and j ≤ n_2 do
6:         if A_1[i] ≤ A_2[j] then
7:             B[k] ← A_1[i]
8:             i ← i + 1
9:         else
10:            B[k] ← A_2[j]
11:            j ← j + 1
12:        end if
13:        k ← k + 1
14:    end while
15:    while i ≤ n_1 do
16:        B[k] ← A_1[i]
17:        i ← i + 1, k ← k + 1
18:    end while
19:    while j ≤ n_2 do
20:        B[k] ← A_2[j]
21:        j ← j + 1, k ← k + 1
22:    end while
23:    return B
24: end function
```

10.2.2 Why the result is sorted, and how much Merge costs

At every step, the next value must be the smaller of the two current elements: all later elements of each input are at least as large as that input's current element. Appending the smaller one therefore keeps B sorted.

Let $n_1 = |A_1|$ be the number of elements in the first array and $n_2 = |A_2|$ the number of elements in the second. Merge copies each of these elements into B exactly once: n_1 copies come from A_1 and n_2 from A_2, for a total of $n_1 + n_2$ copies. The pointers only move forwards, and each copy and pointer update takes constant time. Its running time is therefore

$$\boxed{T_{\text{Merge}}(n_1, n_2) = \Theta(n_1 + n_2).}$$

The separate output array uses $\Theta(n_1 + n_2)$ additional space; the three indices require only constant extra space.

10.3 MergeSort: split, sort the halves, and merge

Merge combines two arrays that are already sorted. To sort an arbitrary array A, we first split it into two halves and sort each half using MergeSort itself. We then pass the two sorted results to Merge.

When can we stop splitting? An array with zero or one element is already sorted, so we return it immediately. This is the base case.

For $n > 1$, let $m = \lceil n/2 \rceil$. Using indices starting at 1, the two halves are

$$A_1 = A[1, \ldots, m], \qquad A_2 = A[m+1, \ldots, n].$$

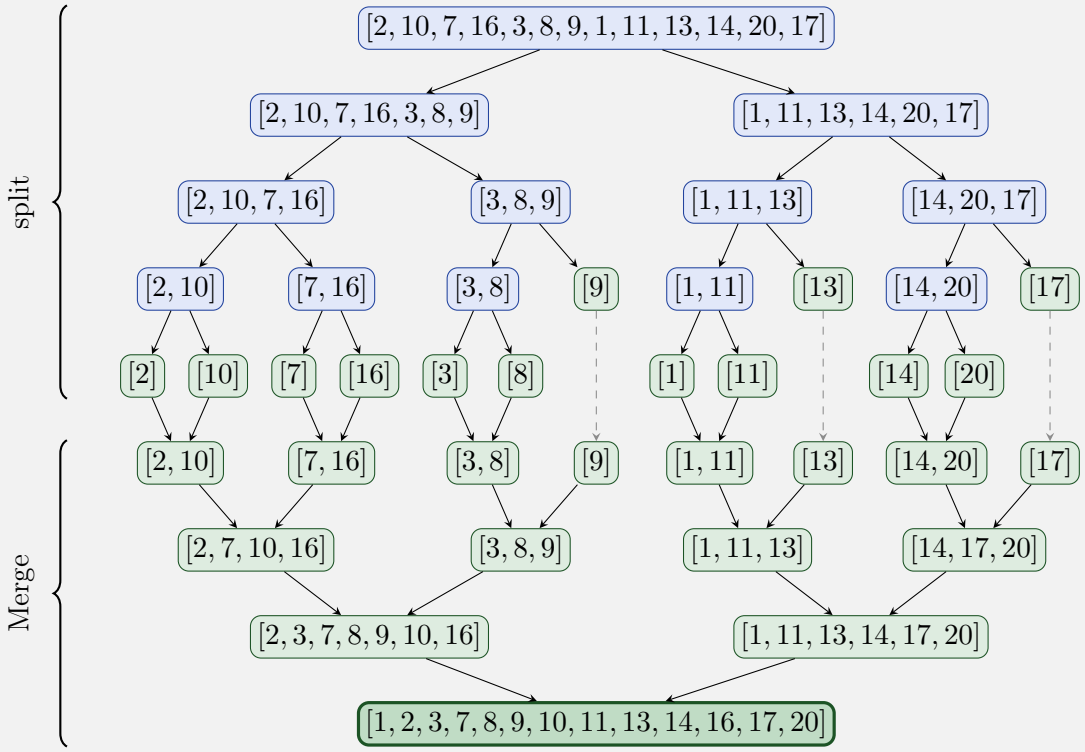
The first half contains m elements and the second contains $n - m$. For our 13-element input, $m = 7$: the halves contain positions 1 through 7 and positions 8 through 13.

Algorithm 10.2: MergeSort

```
1:  function MergeSort(A)
2:      n ← |A|                                   ▷ Θ(1)
3:      if n ≤ 1 then                             ▷ Θ(1)
4:          return A                              ▷ Θ(1)
5:      end if
6:      m ← ⌈n/2⌉                                 ▷ Θ(1)
7:      A_1 ← A[1, ..., m]                        ▷ Θ(m) to copy
8:      A_2 ← A[m + 1, ..., n]                    ▷ Θ(n − m) to copy
9:      A_1 ← MergeSort(A_1)                      ▷ T(m)
10:     A_2 ← MergeSort(A_2)                      ▷ T(n − m)
11:     return Merge(A_1, A_2)                    ▷ Θ(n)
12: end function
```

Example 46: MergeSort on the 13-element array

MergeSort on $A = [2, 10, 7, 16, 3, 8, 9, 1, 11, 13, 14, 20, 17]$. Blue arrays are still unsorted; green arrays are sorted.



10.3.1 Running time: expanding the recurrence

Let $T(n)$ be the number of steps for an array of n elements. Assume first that n is a power of 2, so $m = n - m = n/2$. Counting the steps of Algorithm 10.2 gives a *recurrence*:

$$T(n) = \begin{cases} c_1, & n \leq 1, \\ 2T(n/2) + c_2 n + c_3, & n > 1, \end{cases}$$

Here c_1 is the base-case cost, $c_2 n$ accounts for copying the halves and merging, and c_3 is the remaining constant work. The constants are positive and independent of n.

We solve this recurrence by repeatedly substituting it into itself. As in the earlier recurrence expansions, a term and its replacement are underlined in the same colour.

First expansion. Apply the recurrence to an input of size $n/2$:

$$T(n/2) = 2T(n/4) + c_2\frac{n}{2} + c_3.$$

Substitute this expression for $T(n/2)$ and distribute the outer 2:

$$\begin{aligned} T(n) &= 2\,T(n/2) + c_2 n + c_3 \\ &= 2\left[2T(n/4) + c_2\frac{n}{2} + c_3\right] + c_2 n + c_3 \\ &= 4T(n/4) + c_2 n + 2c_3 + c_2 n + c_3 \\ &= 4T(n/4) + 2c_2 n + 3c_3. \end{aligned}$$

There are now four subproblems of size $n/4$. The two linear contributions add up to $2c_2 n$, and the constant contributions add up to $3c_3$.

Second expansion. Now replace $T(n/4)$ using the same recurrence:

$$\begin{aligned} T(n) &= 4\,T(n/4) + 2c_2 n + 3c_3 \\ &= 4\left[2T(n/8) + c_2\frac{n}{4} + c_3\right] + 2c_2 n + 3c_3 \\ &= 8T(n/8) + c_2 n + 4c_3 + 2c_2 n + 3c_3 \\ &= 8T(n/8) + 3c_2 n + 7c_3. \end{aligned}$$

Third expansion.

$$\begin{aligned} T(n) &= 8\,T(n/8) + 3c_2 n + 7c_3 \\ &= 8\left[2T(n/16) + c_2\frac{n}{8} + c_3\right] + 3c_2 n + 7c_3 \\ &= 16T(n/16) + 4c_2 n + 15c_3. \end{aligned}$$

The general pattern. Each substitution doubles the number of subproblems and halves their size. At each level, the linear work adds up to $c_2 n$:

$$1 \cdot c_2 n = 2 \cdot c_2\frac{n}{2} = 4 \cdot c_2\frac{n}{4} = \cdots = c_2 n.$$

After expanding i levels, we therefore have

$$\boxed{T(n) = 2^i T\left(\frac{n}{2^i}\right) + i\,c_2 n + (2^i - 1)c_3.}$$

Here $2^i T(n/2^i)$ is the work still to be expanded, $i\,c_2 n$ accounts for the i levels already expanded, and the constant work comes from $1 + 2 + 4 + \cdots + 2^{i-1} = 2^i - 1$ calls at those levels.

When do we stop? We reach the base case when each remaining subproblem has size 1:

$$\frac{n}{2^i} = 1 \quad \Longrightarrow \quad 2^i = n \quad \Longrightarrow \quad i = \log_2 n.$$

Substituting $i = \log_2 n$ and $T(1) = c_1$ gives

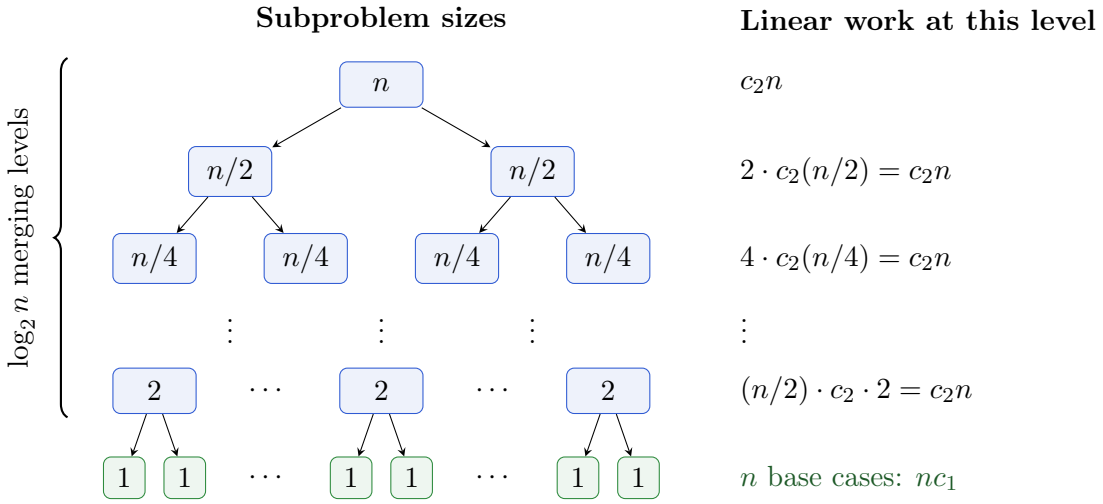
$$\begin{aligned} T(n) &= nT(1) + c_2 n \log_2 n + (n-1)c_3 \\ &= c_1 n + c_2 n \log_2 n + c_3(n-1). \end{aligned}$$

The first and last terms are linear in n. The dominant term is $c_2 n \log_2 n$, so

$$\boxed{T(n) = \Theta(n \log n).}$$

For a general n, the two recursive sizes are $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$. The number of levels is $\lceil \log_2 n \rceil$, and the same asymptotic bound holds.

Visualizing the work at each level. Each box below represents a recursive call and shows the number of elements it sorts. The subarrays at each merging level therefore contain n elements altogether.



Each merging level contributes $c_2 n$ linear work. There are $\log_2 n$ such levels, giving $c_2 n \log_2 n$ altogether. At the bottom, the n single-element inputs contribute nc_1.

11 Randomized Algorithms

11.1 Making random choices inside an algorithm

A deterministic algorithm follows the same execution whenever it receives the same input. We now allow the algorithm to make random choices during its execution. For example, a sorting algorithm may choose a different element to work with first on each run.

Definition 11.1: Randomized algorithm

A *randomized algorithm* uses random choices as part of its computation. Even for a fixed input, different choices may lead to different executions and running times.

For randomized QuickSort, every execution will still return a sorted array. What changes with the random choices is the amount of work needed to obtain it. To describe this cost, we first need to understand an average taken over random outcomes.

11.2 Expected value: averaging over random outcomes

Imagine rolling a fair die many times and averaging the results. Each of the six faces appears with probability 1/6, so each face contributes equally to the average.

Definition 11.2: Random variable and expected value

A discrete *random variable* X assigns a numerical value to each random outcome. Its *expected value* is the weighted average of its possible values:

$$\mathbb{E}[X] = \sum_{x \in \text{range}(X)} x \, \mathbb{P}(X = x).$$

Each value x is weighted by its probability of occurring.

Example 47: One die, then two dice

Step 1: one fair die. Let Y be the number on its upper face. The possible values are $1, 2, 3, 4, 5, 6$, each with probability $1/6$. Thus

$$\begin{aligned}\mathbb{E}[Y] &= 1 \cdot \frac{1}{6} + 2 \cdot \frac{1}{6} + 3 \cdot \frac{1}{6} + 4 \cdot \frac{1}{6} + 5 \cdot \frac{1}{6} + 6 \cdot \frac{1}{6} \\ &= \frac{1+2+3+4+5+6}{6} = \frac{21}{6} = 3.5.\end{aligned}$$

Step 2: the sum of two fair independent dice. Let $X = Y_1 + Y_2$. There are $6 \cdot 6 = 36$ equally likely ordered pairs of faces. The table shows the sum for each pair:

$Y_1 \backslash Y_2$	1	2	3	4	5	6
1	2	3	4	5	6	7
2	3	4	5	6	7	8
3	4	5	6	7	8	9
4	5	6	7	8	9	10
5	6	7	8	9	10	11
6	7	8	9	10	11	12

The sum 2 occurs only for $(1, 1)$, whereas 3 occurs for $(1, 2)$ and $(2, 1)$. The sum 7 occurs for six different pairs. Therefore

$$\mathbb{P}(X = 2) = \frac{1}{36}, \qquad \mathbb{P}(X = 3) = \frac{2}{36}, \qquad \mathbb{P}(X = 7) = \frac{6}{36}.$$

The possible sums are not equally likely. We can compute their weighted average using the definition. A shorter calculation uses *linearity of expectation*: the expected value of a sum is the sum of the expected values. Hence

$$\mathbb{E}[X] = \mathbb{E}[Y_1 + Y_2] = \mathbb{E}[Y_1] + \mathbb{E}[Y_2] = 3.5 + 3.5 = 7.$$

For a randomized algorithm, the running time on a fixed input is also a random variable. Its expected running time averages over the algorithm's random choices.

11.3 QuickSort: choose a pivot and partition

QuickSort sorts an array by choosing one of its elements as a *pivot*. It compares the remaining elements with that pivot and separates them into two smaller arrays.

Definition 11.3: Pivot and partition

Choose an index p uniformly from $\{1, \ldots, n\}$ and let $v = A[p]$ be the pivot value. Excluding $A[p]$, form

$$A_1 = [\text{elements of } A \text{ that are } \leq v], \qquad A_2 = [\text{elements of } A \text{ that are } > v].$$

Every remaining element goes into exactly one of these two arrays. This separation is called *partitioning*.

Now sort A_1 and A_2 recursively. If their sorted results are A_1' and A_2', the answer is their concatenation with the pivot between them:

$$[\, A_1',\ v,\ A_2' \,].$$

Every value in A_1' is at most v, and every value in A_2' is greater than v. Once the two parts are sorted, placing them in this order produces a sorted array. As before, an array with zero or one element is already sorted and is returned immediately.

Example 48: Partitioning around the pivot 11

Start with

$$A = [11, 9, 2, 16, 7, 1, 8, 0, 13, 15, 3].$$

Step 1: choose a pivot. Suppose the random index is $p = 1$, so $v = A[1] = 11$.

Step 2: compare each remaining element with 11. The values $9, 2, 7, 1, 8, 0, 3$ go into A_1, and $16, 13, 15$ go into A_2. Thus

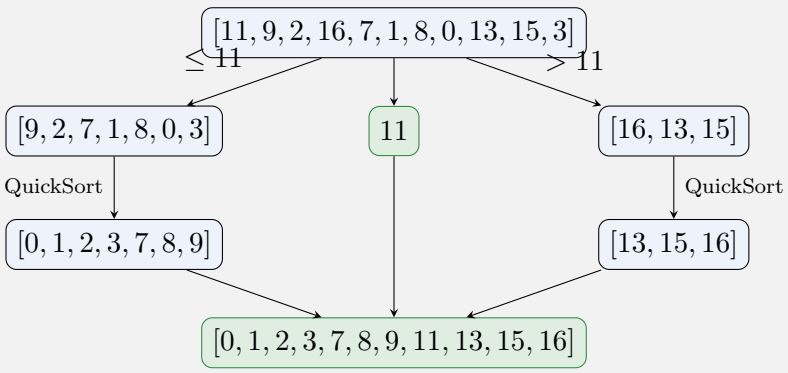
$$A_1 = [9, 2, 7, 1, 8, 0, 3], \qquad A_2 = [16, 13, 15].$$

Step 3: sort the two parts recursively. The calls return

$$A_1' = [0, 1, 2, 3, 7, 8, 9], \qquad A_2' = [13, 15, 16].$$

Step 4: concatenate the results and the pivot.

$$[\, A_1', 11, A_2' \,] = [0, 1, 2, 3, 7, 8, 9, 11, 13, 15, 16].$$



11.3.1 Following the recursive calls

The previous example showed what the two recursive calls return. We can now follow the choices that produce those results.

Example 49: Continuing the same QuickSort execution

Step 1: sort $A_1 = [9, 2, 7, 1, 8, 0, 3]$ using pivot 3. Partitioning gives

$$[2, 1, 0] \quad \boxed{3} \quad [9, 7, 8].$$

Step 2: sort $[2, 1, 0]$ using pivot 1. The partition is

$$[0] \quad \boxed{1} \quad [2].$$

Both recursive inputs have one element, so they are returned as they are. Concatenating gives $[0, 1, 2]$.

Step 3: sort $[9, 7, 8]$ using pivot 7.

The partition is

$$[\,] \quad \boxed{7} \quad [9, 8].$$

The empty array is a base case.

Sorting $[9, 8]$ with pivot 9 gives

$$[8] \quad \boxed{9} \quad [\,],$$

so this recursive call returns $[8, 9]$.

Combining this result with pivot 7 gives

$$[\,] \quad \boxed{7} \quad [8, 9] \quad \longrightarrow \quad [7, 8, 9].$$

Step 4: return the sorted first part. Join the results from Steps 2 and 3 around the pivot from Step 1:

$$[0, 1, 2] \quad \boxed{3} \quad [7, 8, 9] \quad \longrightarrow \quad [0, 1, 2, 3, 7, 8, 9].$$

Step 5: sort the original second part, $A_2 = [16, 13, 15]$. Choose pivot 15. Its partition is $[13] \quad \boxed{15} \quad [16]$, so the call returns $[13, 15, 16]$.

Step 6: return to the original pivot 11. We now have the two sorted results needed to finish:

$$[0, 1, 2, 3, 7, 8, 9] \quad \boxed{11} \quad [13, 15, 16].$$

The final array is $[0, 1, 2, 3, 7, 8, 9, 11, 13, 15, 16]$.

11.3.2 The QuickSort algorithm

Indices start at 1. Appending to the temporary arrays is assumed to take amortized constant time. The final concatenation copies the two sorted parts and the pivot into the returned array.

Algorithm 11.1: Randomized QuickSort

```
1:  function QuickSort(A)
2:      n ← |A|                                              ▷ Θ(1)
3:      if n ≤ 1 then
4:          return A
5:      end if
6:      Choose p uniformly from {1, ..., n}
7:      v ← A[p]
8:      A_1 ← [ ], A_2 ← [ ]
9:      for i = 1, ..., n do                                 ▷ Θ(n) in total
10:         if i ≠ p then
11:             if A[i] ≤ v then
12:                 Append A[i] to A_1
13:             else
14:                 Append A[i] to A_2
15:             end if
16:         end if
17:     end for
18:     A'_1 ← QuickSort(A_1)                                ▷ T(|A_1|)
19:     A'_2 ← QuickSort(A_2)                                ▷ T(|A_2|)
20:     return Concatenate(A'_1, [v], A'_2)                  ▷ Θ(n)
21: end function
```

The test $i \neq p$ excludes the selected pivot from both recursive inputs. Other occurrences of the same value are kept in A_1. Each recursive input has fewer than n elements, so the algorithm eventually reaches its base cases.

11.4 The cost of QuickSort

For the running-time analysis in this section, assume that the n values are distinct. Partitioning compares the pivot with the other $n-1$ elements. If k values are smaller than the pivot, the two recursive sizes are k and $n-1-k$. A call therefore has the form

$$T(n) = T(k) + T(n-1-k) + \Theta(n).$$

The linear term includes partitioning and the final concatenation. The random pivot determines k, so it determines how balanced the two recursive inputs will be.

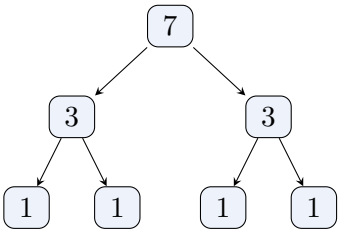
11.4.1 Balanced partitions and the worst case

Balanced partitions. A pivot near the middle of the sorted order leaves two inputs of roughly half the size. Repeated balanced partitions give logarithmic depth, with at most linear work across each level. This leads to $\Theta(n \log n)$ time.

The worst case. If the pivot is always an extreme value (the smallest or the largest), one recursive input is empty and the other has $n-1$ elements. The next nonempty call has size $n-1$, then $n-2$, and so on. The number of pivot comparisons is

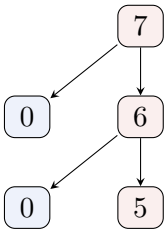
$$(n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2} = \Theta(n^2).$$

Balanced pivots



Subproblem sizes shrink quickly.

Always an extreme pivot



⋮

The nonempty branch loses only one element per call.

When there are $m \geq 2$ distinct values, the chance of picking an extreme value is $2/m$. Consequently, the probability of choosing an extreme pivot at every non-base call is

$$\frac{2}{n} \cdot \frac{2}{n-1} \cdots \frac{2}{2} = \frac{2^{n-1}}{n!}.$$

This probability tends to 0 very quickly as n grows. Randomization allows this bad execution, but does not force it.

11.4.2 The expected cost

Averaging over the random pivot choices, one can prove that the expected running time of QuickSort is $O(n \log n)$:

$$\boxed{\mathbb{E}[T(n)] = O(n \log n), \qquad T_{\text{worst}}(n) = \Theta(n^2).}$$

We do not prove the expected bound here.